

一、基础算法

快速排序算法模板

```
void quick_sort(int q[], int l, int r)
{
    //递归的终止情况
    if (l >= r) return;

    //选取分界线。这里选数组中间那个数
    int i = l - 1, j = r + 1, x = q[l + r >> 1];
    //划分成左右两个部分
    while (i < j)
    {
        do i ++ ; while (q[i] < x);
        do j -- ; while (q[j] > x);
        if (i < j) swap(q[i], q[j]);
    }
    //对左右部分排序
    quick_sort(q, l, j), quick_sort(q, j + 1, r);
}
```

边界问题

因为边界问题只有这两种组合，不能随意搭配

```
x不能取q[l]和q[l+r>>1];
quick_sort(q,l,i-1),quick_sort(q,i,r);
```

```
x不能取q[r]和q[(l+r+1)>>1];
quick_sort(q,l,j),quick_sort(q,j+1,r);
```

归并排序算法模板

```
void merge_sort(int q[], int l, int r)
{
    //递归的终止情况
    if (l >= r) return;
    //第一步：分成子问题
    int mid = l + r >> 1;
    //第二步：递归处理子问题
    merge_sort(q, l, mid);
    merge_sort(q, mid + 1, r);

    //第三步：合并子问题
    int k = 0, i = l, j = mid + 1;
    while (i <= mid && j <= r)
        if (q[i] <= q[j]) tmp[k ++ ] = q[i ++ ];
        else tmp[k ++ ] = q[j ++ ];
}
```

```

while (i <= mid) tmp[k ++ ] = q[i ++ ];
while (j <= r) tmp[k ++ ] = q[j ++ ];
//第四步: 复制回原数组
for (i = l, j = 0; i <= r; i ++, j ++ ) q[i] = tmp[j];
}

```

逆序对数量

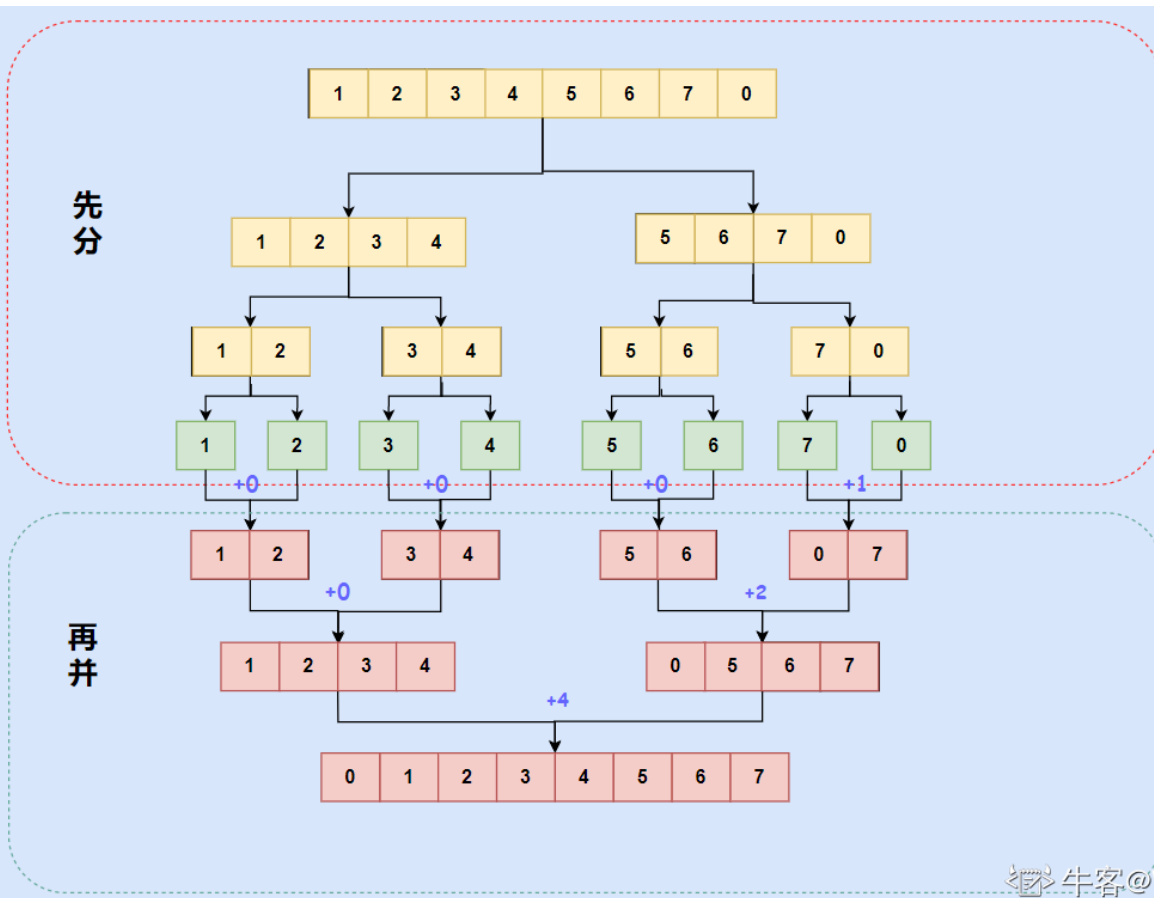
在数组中的两个数字，如果前面一个数字大于后面的数字，则这两个数字组成一个逆序对。输入一个数组，求出这个数组中的逆序对的总数P。

思路：归并排序

举个例子：

在合并 {4,5} {1,2} 的时候，首先我们判断 $1 < 4$ ，我们即可统计出逆序对为2，为什么呢？这利用了数组的部分有序性。因为我们知道 {4,5} 这个数组必然是有序的，因为是合并上来的。此时当 1比4小的时候，证明4以后的数也都比1大，此时就构成了从4开始到 {4,5} 这个数组结束，这么多个逆序对（2个），此时利用一个临时数组，将1存放起来，接着比较2和4的大小，同样可以得到有2个逆序对，于是将2也放进临时数组中，此时右边数组已经完全没有元素了，则将左边剩余的元素全部放进临时元素中，最后将临时数组中的元素放进原数组对应的位置。

最后接着向上合并~



牛客@WosAlan

```

using namespace std;

typedef long long LL;

```

```

const int N = 100010;
int n;
int q[N], tmp[N];
LL merge_sort(int l, int r) {
    if (l >= r) return 0;
    int mid = (l + r) >> 1;
    LL res = merge_sort(l, mid) + merge_sort(mid + 1, r);
    // 归并的过程
    int k = 0, i = l, j = mid + 1;
    while (i <= mid && j <= r)
        if (q[i] <= q[j]) tmp[k++] = q[i++];
        else {
            tmp[k++] = q[j++];
            res += mid - i + 1;
        }
    // 扫尾
    while (i <= mid) tmp[k++] = q[i++];
    while (j <= r) tmp[k++] = q[j++];
    // 物归原主
    for (i = l, j = 0; i <= r; j++, i++) q[i] = tmp[j];
    return res;
}
int main() {
    cin >> n;
    for (int i = 0; i < n; i++) cin >> q[i];
    cout << merge_sort(0, n - 1) << endl;
    return 0;
}

```

整数二分算法模板

对lower_bound来说，它寻找的就是第一个满足条件“值大于等于x”的元素的位置；对upper_bound函数来说，它寻找的是第一个满足“值大于x”的元素的位置。

```

bool check(int x) { /* ... */ } // 检查x是否满足某种性质

// 区间[l, r]被划分成[l, mid]和[mid + 1, r]时使用：
int bsearch_1(int l, int r)
{
    while (l < r)
    {
        int mid = l + r >> 1;
        if (check(mid)) r = mid;    // check()判断mid是否满足性质
        else l = mid + 1; // 左加右减
    }
    return l;
}

// 区间[l, r]被划分成[l, mid - 1]和[mid, r]时使用：
int bsearch_2(int l, int r)
{
    while (l < r)
    {

```

```

        int mid = l + r + 1 >> 1; //如果下方else后面是r则这里加1
        if (check(mid)) l = mid;
        else r = mid - 1; //左加右减
    }
    return l;
}

```

浮点数二分算法模板

```

bool check(double x) { /* ... */ } // 检查x是否满足某种性质

double bsearch_3(double l, double r)
{
    const double eps = 1e-6; // eps 表示精度，取决于题目对精度的要求
    while (r - l > eps)
    {
        double mid = (l + r) / 2;
        if (check(mid)) r = mid;
        else l = mid;
    }
    return l;
}

```

分数的四则运算模板

```

// 分数的标识
struct Fraction {
    int u, d; // up down
};

// 求公约数
int gcd(int a, int b)
{
    return b ? gcd(b, a % b) : a;
}

// 分数的化简
Fraction reduction(Fraction f) {
    // 如为负数，将负号调整到分子
    if (f.d < 0) {
        f.d = abs(f.d);
        f.u = -f.u;
    }
    // 分子为0，将分母置为1
    if (f.u == 0) f.d = 1;
    else { // 分子分母有公约数
        int g = gcd(abs(f.u), f.d);
        if (g != 1) {
            f.d /= g;
            f.u /= g;
        }
    }
    return f;
}

```

```

}
// 分数的打印
void print(Fraction f) {
    f = reduction(f);
    if (f.d == 1)printf("%d", f.u); // 整数
    else if (abs(f.u) > abs(f.d)) { // 假分数
        printf("%d %d/%d", f.u / f.d, abs(f.u) % f.d, f.d);
    } else { //真分数
        printf("%d/%d", f.u, f.d);
    }
    printf("\n");
}

// 分数加法
Fraction add(Fraction a, Fraction b) {
    Fraction c;
    c.u = a.u * b.d + a.d * b.u;
    c.d = a.d * b.d;
    return reduction(c);
}

// 分数减法
Fraction sub(Fraction a, Fraction b) {
    Fraction c;
    c.u = a.u * b.d - a.d * b.u;
    c.d = a.d * b.d;
    return reduction(c);
}

// 分数乘法
Fraction multi(Fraction a, Fraction b) {
    Fraction c;
    c.u = a.u * b.u;
    c.d = a.d * b.d;
    return reduction(c);
}

// 分数的除法
Fraction divide(Fraction a, Fraction b) {
    Fraction c;
    c.u = a.u * b.d;
    c.d = a.d * b.u;
    return reduction(c);
}

```

高精度加法

```

// C = A + B, A >= 0, B >= 0
vector<int> add(vector<int> &a,vector<int> &b){
    //c为答案
    vector<int> c;
    //t为进位
    int t=0;
    for(int i=0;i<a.size()||i<b.size();i++){
        //不超过a的范围添加a[i]
        if(i<a.size())t+=a[i];
    }
}

```

```

        //不超过b的范围添加b[i]
        if(i<b.size())t+=b[i];
        //取当前位的答案
        c.push_back(t%10);
        //是否进位
        t/=10;
    }
    //如果t!=0的话向后添加1
    if(t)c.push_back(1);
    return c;
}

```

高精度减法

```

// C = A - B, 满足A >= B, A >= 0, B >= 0
vector<int> sub(vector<int> &A, vector<int> &B)
{
    //答案
    vector<int> C;
    //遍历最大的数
    for (int i = 0, t = 0; i < A.size(); i ++ )
    {
        //t为进位
        t = A[i] - t;
        //不超过B的范围t=A[i]-B[i]-t;
        if (i < B.size()) t -= B[i];
        //合二为一，取当前位的答案
        C.push_back((t + 10) % 10);
        //借位操作
        if (t < 0) t = 1;
        else t = 0;
    }
    //去除前导零
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}

```

高精度比大小 (cmp函数)

```

//高精度比大小
bool cmp(vector<int> &A, vector<int> &B) {
    if (A.size() != B.size())
        return A.size() > B.size();
    for (int i = A.size() - 1; i >= 0; i -- )
        if (A[i] != B[i])
            return A[i] > B[i];
    return true;
}

```

高精度乘低精度

```
// C = A * b, A >= 0, b >= 0
vector<int> mul(vector<int> &A, int b)
{
    //类似于高精度加法
    vector<int> C;
    //t为进位
    int t = 0;
    for (int i = 0; i < A.size() || t; i++)
    {
        //不超过A的范围t=t+A[i]*b
        if (i < A.size()) t += A[i] * b;
        //取当前位的答案
        C.push_back(t % 10);
        //进位
        t /= 10;
    }
    //去除前导零
    while (C.size() > 1 && C.back() == 0) C.pop_back();

    return C;
}
```

高精度乘高精度

高精度加减乘除: <https://www.bilibili.com/video/BV1LA411v7mt/>

```
vector<int> mul(vector<int> &A, vector<int> &B) {
    vector<int> C(A.size() + B.size() + 10); // 初始化为 0, C的size可以大一点

    for (int i = 0; i < A.size(); i++)
        for (int j = 0; j < B.size(); j++)
            C[i + j] += A[i] * B[j];
    for (int i = 0, t = 0; i < C.size(); i++) { // i = C.size() - 1时 t 一定小于 10
        t += C[i];
        C[i] = t % 10;
        t /= 10;
    }

    while (C.size() > 1 && C.back() == 0) C.pop_back(); // 必须要去前导 0, 因为最高位很可能是 0
    return C;
}
```

高精度除低精度

```
// A / b = C ... r, A >= 0, b > 0
vector<int> div(vector<int> &A, int b, int &r)//高精度A, 低精度b, 余数r
{
    vector<int> C;//答案
    r = 0;
```

```

for (int i = A.size() - 1; i >= 0; i -- )
{
    r = r * 10 + A[i]; //补全r>=b
    C.push_back(r / b); //取当前位的答案
    r %= b; //r%b为下一次计算
}
reverse(C.begin(), C.end()); //倒序为答案
while (C.size() > 1 && C.back() == 0) C.pop_back(); //去除前导零
return C;
}

```

高精度除高精度

高精度加减乘除: <https://www.bilibili.com/video/BV1LA411v7mt/>

```

vector<int> div(vector<int> &A, vector<int> &B, vector<int> &r) {
    vector<int> C;
    if (!cmp(A, B)) {
        C.push_back(0);
        r.assign(A.begin(), A.end());
        return C;
    }
    int j = B.size();
    r.assign(A.end() - j, A.end());
    while (j <= A.size()) {
        int k = 0;
        while (cmp(r, B)) {
            r = sub(r, B);
            k++;
        }
        C.push_back(k);
        if (j < A.size())
            r.insert(r.begin(), A[A.size() - j - 1]);
        if (r.size() > 1 && r.back() == 0)
            r.pop_back();
        j++;
    }
    reverse(C.begin(), C.end());
    while (C.size() > 1 && C.back() == 0)
        C.pop_back();
    return C;
}

```

一维前缀和

前缀和可以用于快速计算一个序列的区间和，也有很多问题里不是直接用前缀和，但是借用了前缀和的思想。

预处理: $s[i] = a[i] + s[i-1]$

求区间 $[l, r]$: $sum = s[r] - s[l-1]$

"前缀和数组"和"原数组"可以合二为一

应用

```
const int N=100010;
int a[N];
int main(){
    int n,m;
    scanf("%d",&n);
    for(int i=1;i<=n;i++)scanf("%d",&a[i]);
    for(int i=1;i<=n;i++)a[i]=a[i-1]+a[i];
    scanf("%d",&m);
    while(m--){
        int l,r;
        scanf("%d%d",&l,&r);
        printf("%d\n",a[r]-a[l-1]);
    }
    return 0;
}
```

二维前缀和

计算矩阵的前缀和: $s[x][y] = s[x-1][y] + s[x][y-1] - s[x-1][y-1] + a[x][y]$

以 $(x1, y1)$ 为左上角, $(x2, y2)$ 为右下角的子矩阵的和为:

计算子矩阵的和: $s = s[x2][y2] - s[x1-1][y2] - s[x2][y1-1] + s[x1-1][y1-1]$

应用

```
int s[1010][1010];

int n,m,q;

int main(){
    scanf("%d%d%d",&n,&m,&q);
    for(int i=1;i<=n;i++)
        for(int j=1;j<=m;j++)
            scanf("%d",&s[i][j]);
    for(int i=1;i<=n;i++)
        for(int j=1;j<=m;j++)
            s[i][j]+=s[i-1][j]+s[i][j-1]-s[i-1][j-1];
    while(q--){
        int x1,y1,x2,y2;
        scanf("%d%d%d%d",&x1,&y1,&x2,&y2);
        printf("%d\n",s[x2][y2]-s[x2][y1-1]-s[x1-1][y2]+s[x1-1][y1-1]);
    }
    return 0;
}
```

一维差分

差分是前缀和的逆运算, 对于一个数组 a , 其差分数组 b 的每一项都是 $a[i]$ 和前一项 $a[i-1]$ 的差。

注意: 差分数组和原数组必须分开存放!!!

给区间 $[l, r]$ 中的每个数加上 c : $B[l] += c, B[r + 1] -= c$

应用

```
using namespace std;
int a[100010], s[100010];

int main(){
    int n, m;
    cin >> n >> m;
    for(int i=1; i<=n; i++) cin >> a[i];
    for(int i=1; i<=n; i++) s[i] = a[i] - a[i-1]; // 读入并计算差分数组
    while(m--){
        int l, r, c;
        cin >> l >> r >> c;
        s[l] += c;
        s[r+1] -= c; // 在原数组中将区间[l, r]加上c
    }
    for(int i=1; i<=n; i++){
        s[i] += s[i-1];
        cout << s[i] << ' ';
    } // 给差分数组计算前缀和, 就求出了原数组
    return 0;
}
```

二维差分

给以 $(x1, y1)$ 为左上角, $(x2, y2)$ 为右下角的子矩阵中的所有元素加上 c :

$s[x1, y1] += c, s[x2 + 1, y1] -= c, s[x1, y2 + 1] -= c, s[x2 + 1, y2 + 1] += c$

应用

```
const int N = 1e3 + 10;
int a[N][N], b[N][N];
void insert(int x1, int y1, int x2, int y2, int c)
{
    b[x1][y1] += c;
    b[x2 + 1][y1] -= c;
    b[x1][y2 + 1] -= c;
    b[x2 + 1][y2 + 1] += c;
}
int main()
{
    int n, m, q;
    cin >> n >> m >> q;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            cin >> a[i][j];
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++)
```

```

        {
            insert(i, j, i, j, a[i][j]); //构建差分数组
        }
    }
    while (q--)
    {
        int x1, y1, x2, y2, c;
        cin >> x1 >> y1 >> x2 >> y2 >> c;
        insert(x1, y1, x2, y2, c); //加c
    }
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++)
        {
            b[i][j] += b[i - 1][j] + b[i][j - 1] - b[i - 1][j - 1]; //二维前缀和
        }
    }
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++)
        {
            printf("%d ", b[i][j]);
        }
        printf("\n");
    }
    return 0;
}

```

关于前缀和 与 差分的相关博客链接: https://blog.csdn.net/qq_39757593/article/details/129219491

位运算

求n的第k位数字: $n \gg k \& 1$

返回n的最后一位1: $\text{lowbit}(n) = n \& -n$

双指针算法

```

for (int i = 0, j = 0; i < n; i ++ )
{
    while (j < i && check(i, j)) j ++ ;
    // 具体问题的逻辑
}

```

常见问题分类:

- (1) 对于一个序列, 用两个指针维护一段区间
- (2) 对于两个序列, 维护某种次序, 比如归并排序中合并两个有序序列的操作

离散化

离散化的本质是建立了一段数列到自然数之间的映射关系 (value -> index), 通过建立新索引, 来缩小目标区间, 使得可以进行一系列连续数组可以进行的操作比如二分, 前缀和等...

离散化首先需要排序去重：

1. 排序：sort(alls.begin(), alls.end())
2. 去重：alls.erase(unique(alls.begin(), alls.end()), alls.end());

```
vector<int> alls; // 存储所有待离散化的值
sort(alls.begin(), alls.end()); // 将所有值排序
alls.erase(unique(alls.begin(), alls.end()), alls.end()); // 去掉重复元素

// 二分求出x对应的离散化的值
int find(int x) // 找到第一个大于等于x的位置
{
    int l = 0, r = alls.size() - 1;
    while (l < r)
    {
        int mid = l + r >> 1;
        if (alls[mid] >= x) r = mid;
        else l = mid + 1;
    }
    return r + 1; // 映射到1, 2, ...n
}
```

应用

```
typedef pair<int, int> PII;

const int N = 300010;

int n, m;
int a[N], s[N];

vector<int> alls; // 存入下标容器
vector<PII> add, query; // add增加容器，存入对应下标和增加的值的的大小
// query存入需要计算下标区间和的容器
int find(int x)
{
    int l = 0, r = alls.size() - 1;
    while (l < r) // 查找大于等于x的最小的值的下标
    {
        int mid = l + r >> 1;
        if (alls[mid] >= x) r = mid;
        else l = mid + 1;
    }
    return r + 1; // 因为使用前缀和，其下标要+1可以不考虑边界问题
}

int main()
{
    cin >> n >> m;
    for (int i = 0; i < n; i++)
    {
        int x, c;
```

```

    cin >> x >> c;
    add.push_back({x, c}); //存入下标即对应的数值c

    alls.push_back(x); //存入数组下标x=add.first
}

for (int i = 0; i < m; i++)
{
    int l, r;
    cin >> l >> r;
    query.push_back({l, r}); //存入要求的区间

    alls.push_back(l); //存入区间左右下标
    alls.push_back(r);
}

// 区间去重
sort(alls.begin(), alls.end());
alls.erase(unique(alls.begin(), alls.end()), alls.end());

// 处理插入
for (auto item : add)
{
    int x = find(item.first); //将add容器的add.seccnd值存入数组a[]当中,
    a[x] += item.second; //在去重之后的下标集合alls内寻找对应的下标并添加数值
}

// 预处理前缀和
for (int i = 1; i <= alls.size(); i++) s[i] = s[i - 1] + a[i];

// 处理询问
for (auto item : query)
{
    int l = find(item.first), r = find(item.second); //在下标容器中查找对应的左右两端[l~r]下
    // 标, 然后通过下标得到前缀和相减再得到区间a[l~r]的和
    cout << s[r] - s[l - 1] << endl;
}

return 0;
}

```

区间合并

```

typedef pair<int, int> PII;
// 将所有存在交集的区间合并
void merge(vector<PII> &segs)
{
    vector<PII> res;

    sort(segs.begin(), segs.end());

    int st = -2e9, ed = -2e9;

```

```

for (auto seg : segs)
    if (ed < seg.first)
    {
        if (st != -2e9) res.push_back({st, ed});
        st = seg.first, ed = seg.second;
    }
    else ed = max(ed, seg.second);

if (st != -2e9) res.push_back({st, ed});

segs = res;
}

```

二、数据结构

单链表

```

const int N=100010;

int head,e[N],ne[N],idx;
//初始化
void init(){
    head=-1;
    idx=0;
}
//在链表头部添加节点
void add_to_head(int x){
    e[idx]=x,ne[idx]=head,head=idx++;
}
//在位置k添加节点x
void add(int k,int x){
    e[idx]=x,ne[idx]=ne[k],ne[k]=idx++;
}
//删除位置k的节点
void remove(int k){
    ne[k]=ne[ne[k]];
}

```

应用

```

int main(){
    int m;
    init();
    cin>>m;
    while(m--){
        int k,x;
        char op;
        cin>>op;
        if(op=='H'){
            cin>>x;
            add_to_head(x);
        }
    }
}

```

```

        }else if(op=='D'){
            cin>>k;
            if(!k)head=ne[head];
            remove(k-1);
        }else {
            cin>>k>>x;
            add(k-1,x);
        }
    }
    for(int i=head;i!=-1;i=ne[i])cout<<e[i]<<' ';
    cout<<endl;
    return 0;
}

```

反转链表

```

void reverse_linklist(){
    if (head == -1) return ;
    int p = head, q = ne[head];
    while (q != -1){
        int o = ne[q];
        ne[q] = p;
        p = q;
        q = o;
    }
    ne[head] = -1;
    head = p;
}

```

双链表

```

const int N=100010;

int e[N],l[N],r[N],idx;

//初始化
void init(){
    l[1]=0;
    r[0]=1;
    idx=2;
}

//在节点a的右边插入一个数x
void insert(int a,int x){
    e[idx]=x;
    l[idx]=a,r[idx]=r[a];
    l[r[a]]=idx,r[a]=idx++;
}

//删除节点a
void remove(int a){
    l[r[a]]=l[a];
}

```

```

    r[l[a]]=r[a];
}

```

应用

```

int main(){
    int m;
    cin>>m;
    init();
    while(m--){
        string op;
        cin>>op;
        int k,x;
        if(op=="L"){//在最左端插入数x
            cin>>x;
            insert(0,x);
        }else if(op=="R"){//在最右端插入数x
            cin>>x;
            insert(l[1],x);
        }else if(op=="D"){//删除第k个插入的数
            cin>>k;
            remove(k+1);
        }else if(op=="IL"){//在第k个位置的左侧插入一个数
            cin>>k>>x;
            insert(l[k+1],x);
        }else if(op=="LR"){//在第k个位置的右侧插入一个数
            cin>>k>>x;
            insert(k+1,x);
        }
    }
    for(int i=r[0];i!=1;i=r[i])printf("%d ",e[i]);
    cout<<endl;
    return 0;
}

```

栈

```

// tt表示栈顶
int stk[N], tt = 0;
// 向栈顶插入一个数
stk[ ++ tt] = x;
// 从栈顶弹出一个数
tt -- ;
// 栈顶的值
stk[tt];
// 判断栈是否为空, 如果 tt > 0, 则表示不为空
if (tt > 0)
{

}

```

应用

```
const int N=100010;
int stk[N],tt;

int main(){
    int m;
    cin>>m;
    while(m--){
        string op;
        int x;
        cin>>op;
        if(op=="push"){
            cin>>x;
            stk[tt++]=x;
        }else if(op=="pop"){
            tt--;
        }else if(op=="query"){
            cout<<stk[tt-1]<<endl;
        }else{
            if(!tt)cout<<"YES"<<endl;
            else cout<<"NO"<<endl;
        }
    }
    return 0;
}
```

表达式求值

```
using namespace std;

stack<int> num;
stack<char> op;

void eval()
{
    auto b = num.top(); num.pop();
    auto a = num.top(); num.pop();
    auto c = op.top(); op.pop();
    int x;
    if (c == '+') x = a + b;
    else if (c == '-') x = a - b;
    else if (c == '*') x = a * b;
    else x = a / b;
    num.push(x);
}

int main()
{
    unordered_map<char, int> pr{{'+', 1}, {'-', 1}, {'*', 2}, {'/', 2}};
    string str;
    cin >> str;
```

```

for (int i = 0; i < str.size(); i ++ )
{
    auto c = str[i];
    if (isdigit(c))
    {
        int x = 0, j = i;
        while (j < str.size() && isdigit(str[j]))
            x = x * 10 + str[j ++ ] - '0';
        i = j - 1;
        num.push(x);
    }
    else if (c == '(') op.push(c);
    else if (c == ')')
    {
        while (op.top() != '(') eval();
        op.pop();
    }
    else
    {
        while (op.size() && op.top() != '(' && pr[op.top()] >= pr[c]) eval();
        op.push(c);
    }
}
while (op.size()) eval();
cout << num.top() << endl;
return 0;
}

```

队列

普通队列

```

// hh 表示队头，tt表示队尾
int q[N], hh = 0, tt = -1;

// 向队尾插入一个数
q[ ++ tt] = x;

// 从队头弹出一个数
hh ++ ;

// 队头的值
q[hh];

// 判断队列是否为空，如果 hh <= tt，则表示不为空
if (hh <= tt)
{

}

```

应用

```

int const N=100010;

int que[N],hh,tt=-1;

int main(){
    int m;
    cin>>m;
    while(m--){
        string op;
        int x;
        cin>>op;
        if(op=="push"){
            cin>>x;
            que[++tt]=x;
        }else if(op=="query"){
            cout<<que[hh]<<endl;
        }else if(op=="pop"){
            hh++;
        }else{
            if(hh>tt)cout<<"YES"<<endl;
            else cout<<"NO"<<endl;
        }
    }
    return 0;
}

```

循环队列

```

// hh 表示队头，tt表示队尾的后一个位置
int q[N], hh = 0, tt = 0;

// 向队尾插入一个数
q[tt ++ ] = x;
if (tt == N) tt = 0;

// 从队头弹出一个数
hh ++ ;
if (hh == N) hh = 0;

// 队头的值
q[hh];

// 判断队列是否为空，如果hh != tt，则表示不为空
if (hh != tt)
{

}

```

单调栈

常见模型：找出每个数左边离它最近的比它大/小的数

```
int tt = 0;
for (int i = 1; i <= n; i ++ )
{
    while (tt && check(stk[tt], i)) tt -- ;
    stk[ ++ tt] = i;
}
```

应用

找出每个数左边离它最近的比它大/小的数

```
stack<int> stk;
int main(){
    int n;
    cin >> n;
    stk.push(-1);
    for (int i = 0; i < n; i ++){
        int x;
        cin >> x;
        while (stk.size() && stk.top() >= x) stk.pop();
        cout << stk.top() << " ";
        stk.push(x);
    }
    return 0;
}
```

单调队列

入队操作：队尾入队，会把之前破坏单调性的元素都从队尾移出（维护单调性）

出队操作：如果队首元素超过区间范围，就将元素从队首出队

元素性质：队首元素，永远是当前维护区间的（最大/最小）值

常见模型：找出滑动窗口中的最大值/最小值

```
int hh = 0, tt = -1;
for (int i = 0; i < n; i ++ )
{
    while (hh <= tt && check_out(q[hh])) hh ++ ; // 判断队头是否滑出窗口
    while (hh <= tt && check(q[tt], i)) tt -- ;
    q[ ++ tt] = i;
}
```

```
const int N = 1000010;
int a[N];

int main()
{
    int n, k;
```

```

cin >> n >> k;
for (int i = 1; i <= n; i++)
    cin >> a[i];

deque<int> q;
// 求滑动窗口最小值
for (int i = 1; i <= n; i++)
{
    // 队列存下标, 维护单调递增
    while (!q.empty() && a[q.back()] > a[i])
        q.pop_back();
    q.push_back(i);
    // 下标小于窗口左边界 i-k, 说明彻底移出窗口
    while (q.front() <= i - k)
        q.pop_front();
    if (i >= k)
        cout << a[q.front()] << " ";
}
q.clear();
cout << "\n";

// 求滑动窗口最大值
for (int i = 1; i <= n; i++)
{
    // 维护单调递减
    while (!q.empty() && a[q.back()] < a[i])
        q.pop_back();
    q.push_back(i);
    while (q.front() <= i - k)
        q.pop_front();
    if (i >= k)
        cout << a[q.front()] << " ";
}
return 0;
}

```

KMP字符串匹配

视频讲解: [最浅显易懂的 KMP 算法讲解哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1AY4y157yL>

博客讲解 (推荐): [详解KMP算法 (上) - LeNotFound 的博客 - 洛谷博客 (luogu.org)]: (<https://lenotfound.blog.luogu.org/xiang-jie-kmp-suan-fa-shang>)

下标从1开始的kmp算法

```

const int N = 100010, M = 1000010;
int n, m;
int ne[N];
char s[M], p[N];
int main()
{
    cin >> n >> p + 1 >> m >> s + 1;
}

```

```

for (int i = 2, j = 0; i <= n; i ++ )
{
    while (j && p[i] != p[j + 1]) j = ne[j];
    if (p[i] == p[j + 1]) j ++ ;
    ne[i] = j;
} //处理ne数组
for (int i = 1, j = 0; i <= m; i ++ )
{
    while (j && s[i] != p[j + 1]) j = ne[j];
    if (s[i] == p[j + 1]) j ++ ;
    if (j == n)
    {
        printf("%d ", i - n);
        j = ne[j];
    }
} //匹配算法
return 0;
}

```

```

// s[]是长文本，p[]是模式串，n是s的长度，m是p的长度
// 求模式串的Next数组：
for (int i = 2, j = 0; i <= m; i ++ )
{
    while (j && p[i] != p[j + 1]) j = ne[j];
    if (p[i] == p[j + 1]) j ++ ;
    ne[i] = j;
}

// 匹配
for (int i = 1, j = 0; i <= n; i ++ )
{
    while (j && s[i] != p[j + 1]) j = ne[j];
    if (s[i] == p[j + 1]) j ++ ;
    if (j == m)
    {
        j = ne[j];
        // 匹配成功后的逻辑
    }
}
}

```

下标从0开始的kmp算法

```

const int N = 1000010;

int n, m;
char s[N], p[N];
int ne[N];

int main()
{
    cin >> m >> p >> n >> s;
    ne[0] = -1;
}

```

```

for (int i = 1, j = -1; i < m; i ++ )
{
    while (j >= 0 && p[j + 1] != p[i]) j = ne[j];
    if (p[j + 1] == p[i]) j ++ ;
    ne[i] = j;
}
for (int i = 0, j = -1; i < n; i ++ )
{
    while (j != -1 && s[i] != p[j + 1]) j = ne[j];
    if (s[i] == p[j + 1]) j ++ ;
    if (j == m - 1)
    {
        cout << i - j << ' ';
        j = ne[j];
    }
}
return 0;
}

```

string版的kmp

```

using namespace std;

int main()
{
    int n, m;
    string s, p;
    cin >> n >> p >> m >> s;

    vector<int> nxt(n); // nxt[j]表示p[0:j]的最长前后匹配缀的长度
    for(int i = 1; i < n; ++i)
    {
        // j可以初始化为nxt[i-1]的值，由于nxt[i-1]是长度且j的坐标从0开始，
        // 所以j直接指向p中i-1的最长前缀后一位，可以直接用来和p[i]尝试匹配
        int j = nxt[i-1];
        // p[i]和p[j]不匹配的话，j指向nxt[j-1]，同理，也可以用来和p[i]进行尝试匹配
        while(j && p[i] != p[j]) j = nxt[j-1];
        if(p[i] == p[j]) j++;
        nxt[i] = j;
    }

    for(int i = 0, j = 0; i < m; ++i)
    {
        while(j && s[i] != p[j]) j = nxt[j-1];
        if(s[i] == p[j]) j++;
        if(j == n)
        {
            cout << i-n+1 << ' ';
            j = nxt[j-1];
        }
    }
    return 0;
}

```

```
}
```

二叉树的存储与遍历

```
const int N = 1e6 + 10;

// 二叉树的存储,l数组为左节点,r数组为右结点
// l[i] 存储的下标为i节点的左节点下标, r[i] 存储的下标为i节点的右节点下标
int l[N], r[N];
// 存储节点的数据
char w[N];
// 节点的下标指针
int idx = 0;

// 先序创建
int pre_create(int n) {
    cin >> w[n];
    if (w[n] == '#') return -1;
    l[n] = pre_create(++idx);
    r[n] = pre_create(++idx);
    return n;
}

// 中序创建
int in_create(int n) {
    if (w[n] == '#') return -1;
    l[n] = in_create(++idx);
    cin >> w[n];
    r[n] = in_create(++idx);
    return n;
}

// 后序创建
int back_create(int n) {
    if (w[n] == '#') return -1;
    l[n] = back_create(++idx);
    r[n] = back_create(++idx);
    cin >> w[n];
    return n;
}

// 先序遍历
void pre_print(int n){
    if (w[n] != '#') cout << w[n] << ' ';
    if (l[n] > 0) pre_print(l[n]);
    if (r[n] > 0) pre_print(r[n]);
}

// 中序遍历
void in_print(int n){
    if (l[n] > 0) in_print(l[n]);
    if (w[n] != '#') cout << w[n] << ' ';
```

```

        if (r[n] > 0) in_print(r[n]);
    }

    // 后序遍历
    void back_print(int n){
        if (l[n] > 0) back_print(l[n]);
        if (r[n] > 0) back_print(r[n]);
        if (w[n] != '#') cout << w[n] << ' ';
    }

    // 层序遍历
    void bfs(int root){
        queue<int> que;
        que.push(root);
        while (!que.empty()) {
            int t = que.front();
            cout << w[t] << ' ';
            que.pop();
            if (l[t] > 0 && w[l[t]] != '#')
                que.push(l[t]);
            if (r[t] > 0 && w[r[t]] != '#')
                que.push(r[t]);
        }
    }
}

```

应用

```

int main(){
    // 先序创建
    pre_create(++idx);
    // 中序创建
    // in_create(++idx);
    // 后序创建
    // back_create(++idx);
    // 先序遍历
    pre_print(1);
    // 中序遍历
    in_print(1);
    // 后序遍历
    back_print(1);
    // 层序遍历
    bfs(1);
    // 测试数据abc##de#g##f###
    // 输出如下:
    // a b c d e g f
    // c b e g d f a
    // c g e f d b a
    // a b c d e f g
    return 0;
}

```

二叉搜索树

```
const int N = 1e5 + 10;

int l[N], r[N];    // l[id]左孩子编号, r[id]右孩子编号
int w[N];          // w[id]节点权值
int idx;           // 节点分配指针

// 创建新节点
inline int newNode(int key)
{
    ++idx;
    w[idx] = key;
    l[idx] = r[idx] = 0;
    return idx;
}

// 插入 key, 返回当前子树根编号
int insert(int i, int key)
{
    if (w[i] == -1)
        return newNode(key);
    if (w[i] == key)
        return i; // 重复值不新增节点
    if (key < w[i])
        l[i] = insert(l[i], key);
    else
        r[i] = insert(r[i], key);
    return i;
}

// 获取以id为根的子树最小值节点编号
int getMin(int id)
{
    while (l[id]) id = l[id];
    return id;
}

// 删除key, 返回更新后的子树根编号
int erase(int i, int key)
{
    if (w[i] == -1) return i;

    if (key < w[i])
        l[i] = erase(l[i], key);
    else if (key > w[i])
        r[i] = erase(r[i], key);
    else
    {
        // 找到待删节点
        if (!l[i]) return r[i];
        if (!r[i]) return l[i];
    }
}
```

```

        // 左右都有孩子：后继替换（右子树最小）
        int succ = getMin(r[i]);
        w[i] = w[succ];
        r[i] = erase(r[i], w[succ]);
    }
    return i;
}

// 重置整棵树
void clear()
{
    memset(w, -1, sizeof w);
    memset(l, 0, sizeof l);
    memset(r, 0, sizeof r);
    idx = 0;
}

// 前序遍历
void pre_print(int n)
{
    if (w[n] == -1) return;
    cout << w[n] << ' ';
    if (l[n]) pre_print(l[n]);
    if (r[n]) pre_print(r[n]);
}

// 中序遍历（BST中序 = 升序序列）
void in_print(int n)
{
    if (w[n] == -1) return;
    if (l[n]) in_print(l[n]);
    cout << w[n] << ' ';
    if (r[n]) in_print(r[n]);
}

// 后序遍历
void after_print(int n)
{
    if (w[n] == -1) return;
    if (l[n]) after_print(l[n]);
    if (r[n]) after_print(r[n]);
    cout << w[n] << ' ';
}

```

应用

```

int main()
{
    clear(); // 初始化
    int root = 0;

```

```

int n;
cin >> n;
for (int i = 0; i < n; i++)
{
    int key;
    cin >> key;
    root = insert(root, key);
}

cout << "前序: ";
pre_print(root);
cout << "\n中序: ";
in_print(root);
cout << "\n后序: ";
after_print(root);

int del;
cout << "\n请输入待删除数值: ";
cin >> del;
root = erase(root, del);
cout << "删除后中序: ";
in_print(root);

return 0;
}

```

Trie树

Trie 树是一种多叉树的结构，每个节点保存一个字符，一条路径表示一个字符串。

相关链接: <https://www.acwing.com/solution/content/27771/>

```

int son[N][26], cnt[N], idx;
// 0号点既是根节点，又是空节点
// son[][] 存储树中每个节点的子节点
// cnt[] 存储以每个节点结尾的单词数量

// 插入一个字符串
void insert(char *str)
{
    int p = 0;
    for (int i = 0; str[i]; i++)
    {
        int u = str[i] - 'a';
        if (!son[p][u]) son[p][u] = ++ idx;
        p = son[p][u];
    }
    cnt[p]++;
}

// 查询字符串出现的次数
int query(char *str)
{

```

```

    int p = 0;
    for (int i = 0; str[i]; i ++ )
    {
        int u = str[i] - 'a';
        if (!son[p][u]) return 0;
        p = son[p][u];
    }
    return cnt[p];
}

```

```

const int N = 100010;

int son[N][26], cnt[N], idx;
char str[N];

void insert(char *str)
{
    int p = 0;
    for (int i = 0; str[i]; i ++ )
    {
        int u = str[i] - 'a';
        if (!son[p][u]) son[p][u] = ++ idx;
        p = son[p][u];
    }
    cnt[p] ++ ;
} //插入

int query(char *str)
{
    int p = 0;
    for (int i = 0; str[i]; i ++ )
    {
        int u = str[i] - 'a';
        if (!son[p][u]) return 0;
        p = son[p][u];
    }
    return cnt[p];
} //查询

int main()
{
    int n;
    scanf("%d", &n);
    while (n -- )
    {
        char op[2];
        scanf("%s%s", op, str);
        if (*op == 'I') insert(str);
        else printf("%d\n", query(str));
    }

    return 0;
}

```

```
}
```

并查集

(1)朴素并查集:

```
int p[N]; //存储每个点的祖宗节点

// 返回x的祖宗节点
int find(int x)
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}

// 初始化, 假定节点编号是1~n
for (int i = 1; i <= n; i++) p[i] = i;

// 合并a和b所在的两个集合:
p[find(a)] = find(b);
```

(2)维护size的并查集:

```
int p[N], size[N];
//p[] 存储每个点的祖宗节点, size[] 只有祖宗节点的有意义, 表示祖宗节点所在集合中的点的数量

// 返回x的祖宗节点
int find(int x)
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}

// 初始化, 假定节点编号是1~n
for (int i = 1; i <= n; i++)
{
    p[i] = i;
    size[i] = 1;
}

// 合并a和b所在的两个集合:
size[find(b)] += size[find(a)];
p[find(a)] = find(b);
```

(3)维护到祖宗节点距离的并查集:

```
int p[N], d[N];
//p[] 存储每个点的祖宗节点, d[x] 存储x到p[x]的距离

// 返回x的祖宗节点
```

```

int find(int x)
{
    if (p[x] != x)
    {
        int u = find(p[x]);
        d[x] += d[p[x]];
        p[x] = u;
    }
    return p[x];
}

// 初始化, 假定节点编号是1~n
for (int i = 1; i <= n; i++)
{
    p[i] = i;
    d[i] = 0;
}

// 合并a和b所在的两个集合:
p[find(a)] = find(b);
d[find(a)] = distance; // 根据具体问题, 初始化find(a)的偏移量

```

应用

```

const int N=100010;
int p[N],n,m;

int find(int x){//找到祖宗节点+路径压缩
    if(p[x]!=x)p[x]=find(p[x]);
    return p[x];
}

int main(){
    scanf("%d%d",&n,&m);
    for(int i=1;i<=n;i++)p[i]=i;
    while(m--){
        char op[2];
        int a,b;
        scanf("%s%d%d",op,&a,&b);
        if(op[0]=='M')p[find(a)]=find(b);
        else {
            if(find(a)==find(b))puts("Yes");
            else puts("No");
        }
    }
    return 0;
}

```

堆

```

// h[N]存储堆中的值, h[1]是堆顶, x的左儿子是2x, 右儿子是2x + 1
// ph[k]存储第k个插入的点在堆中的位置

```

```

// hp[k]存储堆中下标是k的点是第几个插入的
int h[N], ph[N], hp[N], size;

// 交换两个点，及其映射关系
void heap_swap(int a, int b)
{
    swap(ph[hp[a]], ph[hp[b]]);
    swap(hp[a], hp[b]);
    swap(h[a], h[b]);
}

void down(int u)
{
    int t = u;
    if (u * 2 <= size && h[u * 2] < h[t]) t = u * 2;
    if (u * 2 + 1 <= size && h[u * 2 + 1] < h[t]) t = u * 2 + 1;
    if (u != t)
    {
        heap_swap(u, t);
        down(t);
    }
}

void up(int u)
{
    while (u / 2 && h[u] < h[u / 2])
    {
        heap_swap(u, u / 2);
        u >>= 1;
    }
}

// O(n)建堆
for (int i = n / 2; i; i -- ) down(i);

```

应用：堆排序

```

const int N=100010;
int heap[N],cnt;

void down(int u){
    int t=u;
    if(u*2<=cnt&&heap[u*2]<=heap[t])t=u*2;
    if(u*2+1<=cnt&&heap[u*2+1]<=heap[t])t=u*2+1;
    if(t!=u){
        swap(heap[t],heap[u]);
        down(t);
    }
}

//down操作

void up(int u){
    while(u/2&&heap[u/2]>heap[u]){
        swap(heap[u/2],heap[u]);
    }
}

```

```

        u>=1;
    }
} //up操作

int main(){
    int n,m;
    scanf("%d%d",&n,&m);
    for(int i=1;i<=n;i++)scanf("%d",&heap[i]);
    cnt=n;
    for(int i=n/2;i;i--)down(i);
    while(m--){
        printf("%d ",heap[1]);
        heap[1]=heap[cnt--];
        down(1);
    }
    return 0;
}

```

一般hash

(1) 拉链法

```

int h[N], e[N], ne[N], idx;

// 向哈希表中插入一个数
void insert(int x)
{
    int k = (x % N + N) % N;
    e[idx] = x;
    ne[idx] = h[k];
    h[k] = idx ++ ;
}

// 在哈希表中查询某个数是否存在
bool find(int x)
{
    int k = (x % N + N) % N;
    for (int i = h[k]; i != -1; i = ne[i])
        if (e[i] == x)
            return true;

    return false;
}

```

(2) 开放寻址法

```

int h[N];

// 如果x在哈希表中，返回x的下标；如果x不在哈希表中，返回x应该插入的位置
int find(int x)
{
    int t = (x % N + N) % N;
    while (h[t] != null && h[t] != x)
    {

```

```

        t ++ ;
        if (t == N) t = 0;
    }
    return t;
}

```

字符串哈希

核心思想：将字符串看成P进制数，P的经验值是131或13331，取这两个值的冲突概率低

小技巧：取模的数用 2^{64} ，这样直接用unsigned long long存储，溢出的结果就是取模的结果

```

typedef unsigned long long ULL;
ULL h[N], p[N]; // h[k]存储字符串前k个字母的哈希值, p[k]存储  $P^k \bmod 2^{64}$ 

// 初始化
p[0] = 1;
for (int i = 1; i <= n; i ++ )
{
    h[i] = h[i - 1] * P + str[i];
    p[i] = p[i - 1] * P;
}

// 计算子串 str[l ~ r] 的哈希值
ULL get(int l, int r)
{
    return h[r] - h[l - 1] * p[r - l + 1];
}

```

STL

C++ 标准模板库 Standard Template Library，简称 STL。STL 库：STL 容器 和 STL 算法。

名称	容器	头文件	创建	添加	删除	访问
变长数组	vector	<vector>	vector<int> v	v.push_back(x) $O(1)$	v.pop_back() $O(1)$	v[i] $O(1)$
双端队列	deque	<deque>	deque<int> d	d.push_front(x) $O(1)$ d.push_back(x) $O(1)$	d.pop_front() $O(1)$ d.pop_back() $O(1)$	d.front() $O(1)$ d.back() $O(1)$
栈	stack	<stack>	stack<int> s	s.push(x) $O(1)$	s.pop() $O(1)$	s.top() $O(1)$
队列	queue	<queue>	queue<int> q	q.push(x) $O(1)$	q.pop() $O(1)$	q.front() $O(1)$
优先队列 (大根堆)	priority_queue	<queue>	priority_queue<int> q	q.push(x) $O(\log n)$	q.pop() $O(\log n)$	q.top() $O(1)$
有序集合	set	<set>	set<int> s	s.insert(x) $O(\log n)$	s.erase(it) $O(\log n)$	s.count(x) $O(k + \log n)$
无序集合	unordered_set	<unordered_set>	unordered_set<int> s	s.insert(x) $O(1)/O(n)$	s.erase(it) $O(1)/O(n)$	s.count(x) $O(1)/O(n)$
有序键值对映射	map	<map>	map<string, int> h	h[str]=x $O(\log n)$	h.erase(it) $O(\log n)$	h.count(str); h[str] $O(\log n)$
无序键值对映射	unordered_map	<unordered_map>	unordered_map<string, int> h	h[str]=x $O(1)/O(n)$	h.erase(it) $O(1)/O(n)$	h.count(str); h[str] $O(1)/O(n)$

视频讲解：[100 STL 容器哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1tF411G73c/>

vector, 变长数组, 倍增的思想

size() 返回元素个数

empty() 返回是否为空

clear() 清空

front()/back()

push_back()/pop_back()

begin()/end()

[]

vector<int>::iterator iter = std::find(r.begin(), r.end(), target); 查找目标值为target的指针, 不存在返回r.end();

支持比较运算, 按字典序

pair<int, int>

first, 第一个元素

second, 第二个元素

支持比较运算, 以first为第一关键字, 以second为第二关键字 (字典序)

string, 字符串

size()/length() 返回字符串长度

empty()

clear()

substr(起始下标, (子串长度)) 返回子串

c_str() 返回字符串所在字符数组的起始地址

queue, 队列

size()

empty()

push() 向队尾插入一个元素

front() 返回队头元素

back() 返回队尾元素

pop() 弹出队头元素

priority_queue, 优先队列, 默认是大根堆

size()

empty()

push() 插入一个元素

top() 返回堆顶元素

pop() 弹出堆顶元素

定义成小根堆的方式: priority_queue<int, vector<int>, greater<int>> q;

stack, 栈

size()

empty()

push() 向栈顶插入一个元素

top() 返回栈顶元素

pop() 弹出栈顶元素

deque, 双端队列

size()

empty()

clear()

```
front()/back()
push_back()/pop_back()
push_front()/pop_front()
begin()/end()
[]
```

set, map, multiset, multimap, 基于平衡二叉树（红黑树），动态维护有序序列

```
size()
empty()
clear()
begin()/end()
++, -- 返回前驱和后继，时间复杂度  $O(\log n)$ 
```

set/multiset

```
insert() 插入一个数
find() 查找一个数
count() 返回某一个数的个数
erase()
    (1) 输入是一个数x，删除所有x  $O(k + \log n)$ 
    (2) 输入一个迭代器，删除这个迭代器
lower_bound()/upper_bound()
    lower_bound(x) 返回大于等于x的最小的数的迭代器
    upper_bound(x) 返回大于x的最小的数的迭代器
```

map/multimap

```
insert() 插入的数是一个pair
erase() 输入的参数是pair或者迭代器
find()
[] 注意multimap不支持此操作。 时间复杂度是  $O(\log n)$ 
lower_bound()/upper_bound()
```

unordered_set, unordered_map, unordered_multiset, unordered_multimap, 哈希表

和上面类似，增删改查的时间复杂度是 $O(1)$

不支持 lower_bound()/upper_bound(), 迭代器的++, --

bitset, 压位

```
bitset<10000> s;
~, &, |, ^
>>, <<
==, !=
[]
```

count() 返回有多少个1

any() 判断是否至少有一个1

none() 判断是否全为0

set() 把所有位置成1

set(k, v) 将第k位变成v

reset() 把所有位变成0

flip() 等价于~

flip(k) 把第k位取反

三、搜索与图论

树与图的存储

树是一种特殊的图，与图的存储方式相同。
对于无向图中的边 ab ，存储两条有向边 $a \rightarrow b$, $b \rightarrow a$ 。
因此我们可以只考虑有向图的存储。

邻接矩阵

邻接矩阵： $g[a][b]$ 存储边 $a \rightarrow b$ 的距离

邻接表

```
// 对于每个点k，开一个单链表，存储k所有可以走到的点。h[k]存储这个单链表的头结点
int h[N], e[N], ne[N], idx;
// 添加一条边a->b
void add(int a, int b)
{
    // 存下b的值，b下一个指向a的下一个节点，a的下一个节点指向b
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
}
// 初始化
idx = 0;
memset(h, -1, sizeof h);
```

树与图的遍历

时间复杂度 $O(n+m)$ ， n 表示点数， m 表示边数

深度优先遍历

```
int dfs(int u)
{
    st[u] = true; // st[u] 表示点u已经被遍历过

    for (int i = h[u]; i != -1; i = ne[i])
    {
        int j = e[i];
        if (!st[j]) dfs(j);
    }
}
```

DFS代码框架

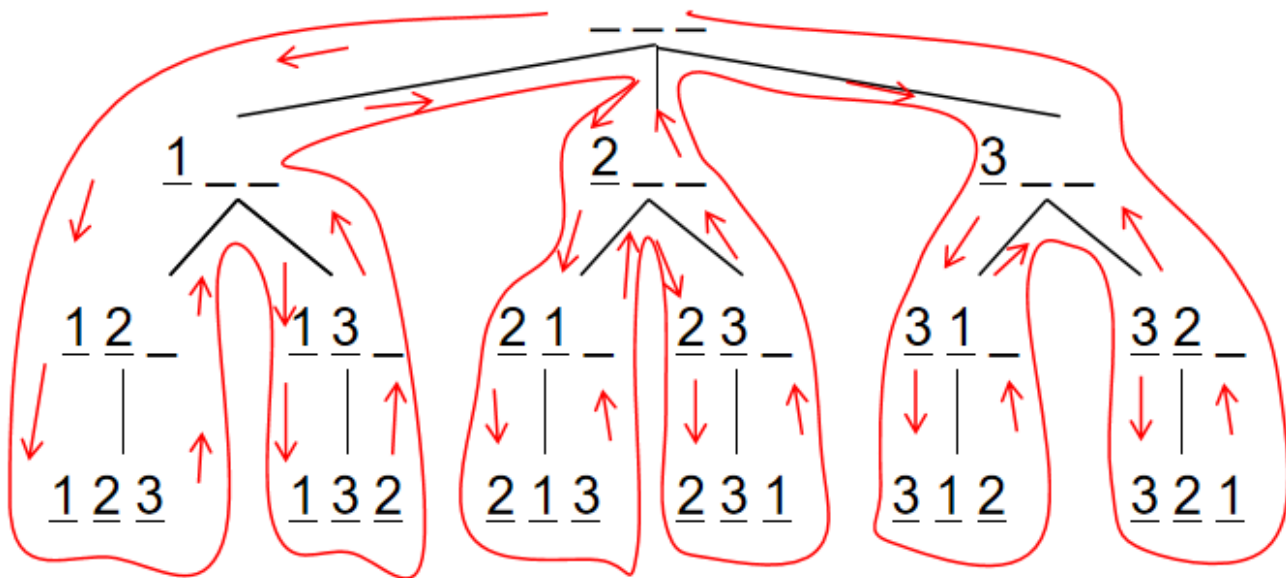
```
ans; // 答案，用全局变量表示
void dfs(层数, 其它参数){
    if(出局判断){ // 到达最底层，或者满足条件退出
        更新答案; // 答案一般用全局变量表示
        return; // 返回到上一层
    }
```

```

}
(剪枝) // 在进一步DFS之前剪枝
for (枚举下一层可能的情况) // 对每个情况继续DFS
    if (used[i] == 0){ // 如果状态i没有用过，就可以进入下一层
        used[i] = 1; // 标记状态i,表示已经用过，在更底层不能再使用
        dfs(层数 + 1, 其它参数); // 下一层
        used[i] = 0; // 恢复状态，回溯时不影响上一层对这个状态的使用
    }
return; // 返回到上一层
}

```

应用：数字全排列



```

#include <iostream>

using namespace std;

int res[10], b[10], n;

void dfs(int k){
    if(k==n){ //k==n则输出n个数字
        for(int i=0; i<n; i++) printf("%d ", res[i]);
        cout<<endl;
    }
    for(int i=1; i<=n; i++){
        if(!b[i]){ //判断是否被用过
            res[k]=i; //当前k位存入位置
            b[i]=1; //表示被占用
            dfs(k+1);
            b[i]=0; //恢复现场
        }
    }
}

```

```
int main(){
    cin>>n;
    dfs(0);//从0开始枚举
    return 0;
}
```

应用：树的重心

给定一颗树，树中包含 n 个结点（编号 $1 \sim n$ ）和 $n - 1$ 条无向边。

请你找到树的重心，并输出将重心删除后，剩余各个连通块中点数的最大值。

重心定义：重心是指树中的一个结点，如果将这个点删除后，剩余各个连通块中节点个数的最大值最小，那么这个节点被称为树的重心。

```
using namespace std;

const int N = 100010, M = N * 2; // 无向图n条边时，最多2n个idx，因为每条边在邻接表中会出现两次

int n; // n个结点, n-1条边
int h[N], e[M], ne[M], idx; // n个链表头, e每一个结点的值, ne每一个结点的next指针
int ans = N; // 最小的最大值
bool st[N]; // 状态数组，防止子节点搜索父节点

void add(int a, int b) // a->b
{
    // e记录当前点的值(地址->值), ne下一点的地址(地址->地址), h记录指向的第一个点的地址(值->地址)
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
} // 头插法

int dfs(int u) // 通过h数组找到子结点的向
{
    st[u] = true; // st标记当前点被搜过

    int size = 0, sum = 0;
    // size删掉元素后各个子连通块的最大值
    // sum当前子树大小，遍历叶节点时，返回1

    for (int i = h[u]; i != -1; i = ne[i]) // 遍历单链表，链表末端初始化为-1
    {
        int j = e[i];
        if (st[j]) continue; // 此处防逆向dfs
        int s = dfs(j); // s各个子连通块的个数
        size = max(size, s); // size删掉元素后各个连通块的最大值
        sum += s; // 各个子连通块个数之和
    }

    size = max(size, n - sum - 1); // 取最大子连通块与父连通块的最大值
    ans = min(ans, size); // 全局变量ans存最小的最大值
    // 注意：本题若求最大的最大值，则只需去除任意叶节点即可，即n-1
    return sum + 1; // 各个子连通块，当前节点之和
}

int main()
```

```

{
    scanf("%d", &n);

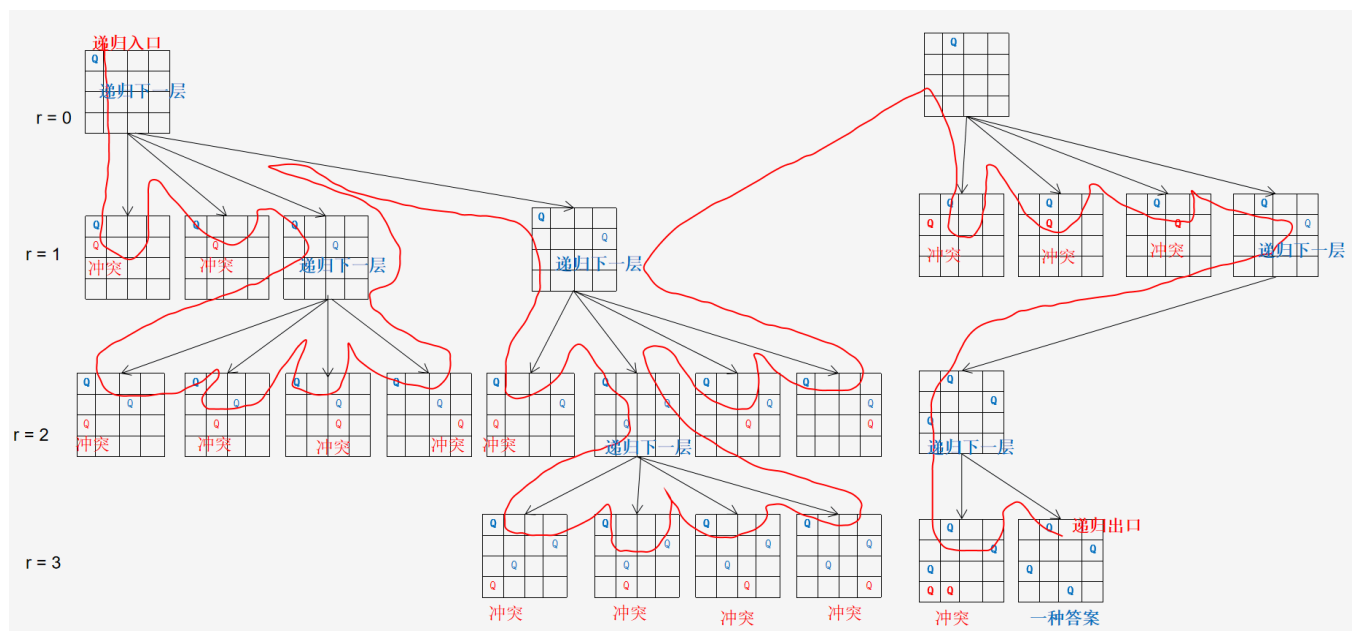
    memset(h, -1, sizeof h); // n个头节点全部指向-1

    for (int i = 0; i < n - 1; i ++ ) // n个结点, n-1条边
    {
        int a, b;
        scanf("%d%d", &a, &b);
        add(a, b), add(b, a); // 不知道子节点还是父节点, 所以需要建两条边可以双向查找
    }

    dfs(1); // 结点编号为1~n且可能只有一个结点, 则参数只能为1
    printf("%d\n", ans);
    return 0;
}

```

应用: n-皇后问题



```

using namespace std;

const int N = 11;

char q[N][N]; // 存储棋盘
bool dg[N * 2], udg[N * 2], cor[N]; // 点对应的两个斜线以及列上是否有皇后

int n;

void dfs(int r)
{
    if (r == n) // 放满了棋盘, 输出棋盘
    {
        for (int i = 0; i < n; i ++ )
        {
            for (int j = 0; j < n; j ++ )

```

```

        cout << q[i][j];
        cout << endl;
    }
    cout << endl;
    return;
}

for(int i = 0; i < n; i++)//第 r 行，第 i 列 是否放皇后
{
    if(!cor[i] && !dg[i + r] && !udg[n - i + r])//不冲突，放皇后
    {
        q[r][i] = 'Q';
        cor[i] = dg[i + r] = udg[n - i + r] = 1;//对应的 列， 斜线 状态改变
        dfs(r + 1);//处理下一行
        cor[i] = dg[i + r] = udg[n - i + r] = 0;//恢复现场
        q[r][i] = '.';
    }
}
}

int main()
{
    cin >> n;
    for (int i = 0; i < n; i ++ )
        for (int j = 0; j < n; j ++ )
            q[i][j] = '.';
    dfs(0);
    return 0;
}

```

宽度优先遍历

```

queue<int> q;
st[1] = true; // 表示1号点已经被遍历过
q.push(1); // 1号点入队

while (q.size())
{
    // 移出节点
    int t = q.front();
    q.pop();
    // 获取被移出的节点的下一层所有节点
    for (int i = h[t]; i != -1; i = ne[i])
    {
        // 获取节点值
        int j = e[i];
        // 将下一层所有未遍历过的节点入队列
        if (!st[j])
        {
            st[j] = true; // 表示点j已经被遍历过
            q.push(j);
        }
    }
}

```

```

    }
}

```

应用：走迷宫

```

typedef pair<int,int> PII;// 声明pair时候必须要在代码前面写上using namespace std;

const int N=110;
int g[N][N],f[N][N],n,m;

int bfs(int x,int y){
    queue<PII> que;
    que.push({x,y});
    int dx[4]={0,1,0,-1},dy[4]={1,0,-1,0};
    while(!que.empty()){
        PII t=que.front();
        que.pop();
        g[t.first][t.second]=1;
        for(int i=0;i<4;i++){
            int a=t.first+dx[i],b=t.second+dy[i];
            if(a>=0&&b>=0&&a<n&&b<m&&!g[a][b]){
                g[a][b]=1;
                f[a][b]=f[t.first][t.second]+1;
                que.push({a,b});
            }
        }
    }
    return f[n-1][m-1];
}

int main(){
    scanf("%d%d",&n,&m);
    for(int i=0;i<n;i++)
        for(int j=0;j<m;j++)
            scanf("%d",&g[i][j]);
    cout<<bfs(0,0)<<endl;
    return 0;
}

```

应用：八数码

```

using namespace std;

int bfs(string state) {
    queue<string> q;
    unordered_map<string, int> d;

    int dx[4] = {-1, 0, 1, 0}, dy[4] = {0, 1, 0, -1};
    string ed = "12345678x";
    q.push(state);
    d[state] = 0;
}

```

```

while (q.size()) {
    auto t = q.front();
    q.pop();
    if (t == ed) //等于结果就输出步数
        return d[t];
    int distance = d[t];
    int k = t.find('x'); //寻找x
    int x = k / 3, y = k % 3; //计算下标
    for (int i = 0; i < 4; i++) {
        int a = x + dx[i], b = y + dy[i];
        if (a >= 0 && a < 3 && b >= 0 && b < 3) {
            swap(t[a * 3 + b], t[k]); //交换
            if (!d.count(t)) { //不存在就入队
                d[t] = distance + 1;
                q.push(t);
            }
            swap(t[a * 3 + b], t[k]); //还原
        }
    }
}
return -1;
}

int main() {
    char s[2];
    string state;
    for (int i = 0; i < 9; i++) {
        cin >> s;
        state += *s;
    }
    cout << bfs(state) << endl;
    return 0;
}

```

拓扑排序

啥是拓扑排序？

一个**有向图**，如果图中有入度为 0 的点，就把这个点删掉，同时也删掉这个点所连的边。

一直进行上面出处理，如果所有点都能被删掉，则这个图可以进行拓扑排序。

拓扑排序讲解：<https://www.bilibili.com/video/BV1XV411X7T7>

纯净版

```

bool topsort()
{
    int hh = 0, tt = -1;

    // d[i] 存储点i的入度
    for (int i = 1; i <= n; i++)
        if (!d[i])

```

```

        q[ ++ tt] = i;

while (hh <= tt)
{
    int t = q[hh ++ ];

    for (int i = h[t]; i != -1; i = ne[i])
    {
        int j = e[i];
        if (-- d[j] == 0)
            q[ ++ tt] = j;
    }
}

// 如果所有点都入队了，说明存在拓扑序列；否则不存在拓扑序列。
return tt == n - 1;
}

```

解说版

```

using namespace std;
const int N = 100010;
int e[N], ne[N], idx; //邻接表存储图
int h[N]; //邻接表的每个头链表
int q[N], hh = 0, tt = -1; //队列保存入度为0的点，也就是能够输出的点
int n, m; //保存图的点数和边数
int d[N]; //保存各个点的入度

void add(int a, int b) {
    e[idx] = b, ne[idx] = h[a], h[a] = idx++;
}

void topsort() {
    for (int i = 1; i <= n; i++) { //遍历一遍顶点的入度。
        if (!d[i]) //如果入度为0，则可以入队列
            q[++tt] = i;
    }
    while (tt >= hh) { //循环处理队列中点的
        int a = q[hh++];
        for (int i = h[a]; i != -1; i = ne[i]) {
            int b = e[i]; //a 有一条边指向b
            d[b]--; //删除边后，b的入度减1
            if (!d[b]) //如果b的入度减为 0,则 b 可以输出，入队列
                q[++tt] = b;
        }
    }
    if (tt == n - 1) { //如果队列中的点的个数与图中点的个数相同，则可以进行拓扑排序
        for (int i = 0; i < n; i++) //队列中保存了所有入度为0的点，依次输出
            printf("%d ", q[i]);
    } else //如果队列中的点的个数与图中点的个数不相同，则可以进行拓扑排序
        cout << -1;
}

```

```

int main() {
    cin >> n >> m; //保存点的个数和边的个数
    memset(h, -1, sizeof h); //初始化邻接矩阵
    while (m--) { //依次读入边
        int a, b;
        cin >> a >> b;
        d[b]++; //顶点b的入度+1
        add(a, b); //添加到邻接矩阵
    }
    topsort(); //进行拓扑排序
    return 0;
}

```

Dijkstra算法

本质Dijkstra算法是贪心

Dijkstra的流程描述

一共有 n 个点，设长度为 n 的 $dist$ 数组，长度为 n 的 st 集合， $dist$ 数组初始化为正无穷，把起始点的 $dist$ 数组设为0，迭代 n 次，每次从数组中选出不在集合中，且距离最短的点，把该点加入 st 集合，用该点当做中转节点去更新所有邻居的最短路。迭代 n 次后，即可求出从起始点到任意点的最短距离。

图解Dijkstra算法：<https://www.bilibili.com/video/BV1uT4y1p7Jy>

朴素版

时间复杂度是 $O(n^2 + m)$ ， n 表示点数， m 表示边数

```

int g[N][N]; // 存储每条边
int dist[N]; // 存储1号点到每个点的最短距离
bool st[N]; // 存储每个点的最短路是否已经确定

// 求1号点到n号点的最短路，如果不存在则返回-1
int dijkstra()
{
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;

    for (int i = 0; i < n - 1; i++)
    {
        int t = -1; // 每次在还未确定最短路的点中，寻找距离最小的点
        for (int j = 1; j <= n; j++)
            if (!st[j] && (t == -1 || dist[t] > dist[j]))
                t = j;

        // 用t更新其他点的距离
        for (int j = 1; j <= n; j++)
            dist[j] = min(dist[j], dist[t] + g[t][j]);

        st[t] = true;
    }
}

```

```

    if (dist[n] == 0x3f3f3f3f) return -1;
    return dist[n];
}

```

应用

```

const int N = 510, M = 100010;

int h[N], e[M], ne[M], w[M], idx; //邻接表存储图
int state[N]; //state 记录是否找到了源点到该节点的最短距离
int dist[N]; //dist 数组保存源点到其余各个节点的距离
int n, m; //图的节点个数和边数

void add(int a, int b, int c) //插入边
{
    e[idx] = b, w[idx] = c, ne[idx] = h[a], h[a] = idx++;
}

void Dijkstra()
{
    memset(dist, 0x3f, sizeof(dist)); //dist 数组的各个元素为无穷大
    dist[1] = 0; //源点到源点的距离为置为 0
    for (int i = 0; i < n; i++)
    {
        int t = -1;
        for (int j = 1; j <= n; j++) //遍历 dist 数组，找到没有确定最短路径的节点中距离源点最近的点 t
        {
            if (!state[j] && (t == -1 || dist[j] < dist[t]))
                t = j;
        }

        state[t] = 1; //state[i] 置为 1。

        for (int j = h[t]; j != -1; j = ne[j]) //遍历 t 所有可以到达的节点 i
        {
            int i = e[j];
            dist[i] = min(dist[i], dist[t] + w[j]); //更新 dist[j]
        }
    }
}

int main()
{
    memset(h, -1, sizeof(h)); //邻接表初始化

    cin >> n >> m;
    while (m--) //读入 m 条边
    {
        int a, b, w;
        cin >> a >> b >> w;
    }
}

```

```

        add(a, b, w);
    }

    Dijkstra();
    if (dist[n] != 0x3f3f3f3f)//如果dist[n]被更新了，则存在路径
        cout << dist[n];
    else
        cout << "-1";
}

```

堆优化版

时间复杂度 $O(m\log n)$, n 表示点数, m 表示边数

```

typedef pair<int, int> PII;

int n;          // 点的数量
int h[N], w[N], e[N], ne[N], idx;          // 邻接表存储所有边
int dist[N];      // 存储所有点到1号点的距离
bool st[N];       // 存储每个点的最短距离是否已确定

// 求1号点到n号点的最短距离，如果不存在，则返回-1
int dijkstra(){
    memset(dist, 0x3f, sizeof dist); // 距离初始化为无穷大
    dist[1] = 0; // 1->1的节点距离为0
    priority_queue<PII, vector<PII>, greater<PII>> heap; // 小根堆
    heap.push({0, 1}); // 插入距离和节点编号

    while(heap.size()){
        auto t = heap.top(); // 取距离源点最近的点
        heap.pop();

        int ver = t.second, distance = t.first; // ver: 节点编号, distance 源点距离ver
        if(st[ver]) continue; // 如果距离已经确定，则跳过该点
        st[ver] = true;
        for(int i = h[ver]; i != -1; i = ne[i]) // 更新ver所指向的节点距离
        {
            int j = e[i];
            if(dist[j] > distance + w[i]){
                dist[j] = distance + w[i];
                heap.push({dist[j], j}); // 距离变小，则入堆
            }
        }
    }
    if(dist[n] == 0x3f3f3f3f) return -1;
    return dist[n];
}

```

关于Dijkstra的相关博客链接：

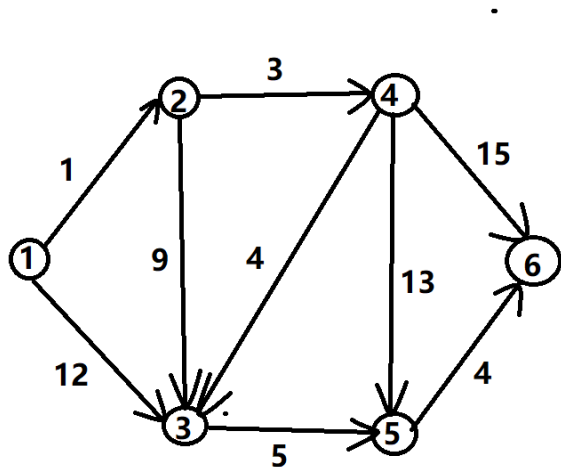
[AcWing 849. Dijkstra求最短路 I: 图解 详细代码 (图解) - AcWing: <https://www.acwing.com/solution/content/38318/>

Bellman-Ford算法

本质Bellman-Ford算法是动态规划

时间复杂度 $O(nm)$, n 表示点数, m 表示边数

注意在模板题中需要对下面的模板稍作修改, 加上备份数组, 详情见模板题。



求出从1号点到n号点的最多经过k条边的最短距离

设 $k=5, n=6$

k \ n	1	2	3	4	5	6
0	0	∞	∞	∞	∞	∞
1	0	1	12	∞	∞	∞
2	0	1	10	4	17	∞
3	0	1	8	4	15	19
4	0	1	8	4	13	19
5	0	1	8	4	13	17

dist数组初始化

本质Bellman-ford算法是动态规划

QQ: 946808247

上图为Bellman-ford草稿图

```
int n, m;           // n表示点数, m表示边数
int dist[N], backup[N]; // dist[x] 存储1到x的最短路距离

struct Edge         // 边, a表示出点, b表示入点, w表示边的权重
{
    int a, b, w;
} edges[M];

// 求1到n的最短路距离, 如果无法从1走到n, 则返回-1。
int bellman_ford()
{
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;

    // 如果第n次迭代仍然会松弛三角不等式, 就说明存在一条长度是n+1的最短路径, 由抽屉原理, 路径中至少存在两个
    // 相同的点, 说明图中存在负权回路。
    for (int i = 0; i < n; i++)
    {
        memcpy(backup, dist, sizeof dist);
        for (int j = 0; j < m; j++)
        {
            int a = edges[j].a, b = edges[j].b, w = edges[j].w;
            if (dist[b] > backup[a] + w)
                dist[b] = backup[a] + w;
        }
    }
}
```

```

    }

    if (dist[n] > 0x3f3f3f3f / 2) return -1;
    return dist[n];
}

```

应用

```

int n,m,k;
const int N=512,M=10012;
struct Edge{
    int a,b,w;
}e[M];
int dist[N];
int back[N];
void bellman_ford(){
    memset(dist,0x3f,sizeof dist);
    dist[1]=0;
    for(int i=0;i<k;i++){
        memcpy(back,dist,sizeof dist);
        for(int j=0;j<m;j++){
            int a=e[j].a,b=e[j].b,c=e[j].w;
            dist[b]=min(dist[b],back[a]+c);
        }
    }
}

int main(){
    scanf("%d%d%d",&n,&m,&k);
    for(int i=0;i<m;i++){
        int a,b,w;
        scanf("%d%d%d",&a,&b,&w);
        e[i]={a,b,w};
    }
    bellman_ford();
    if(dist[n]>0x3f3f3f3f/2)cout<<"impossible"<<endl;
    else cout<<dist[n]<<endl;
    return 0;
}

```

问题：为什么把每一条边用不等式刷k次就是k条件下的值？

你可以想象这个图是1->2->3->4....->n这样一条直线。比如说第一次迭代，为什么只有与原点相连的点才能被更新dist呢？因为原点的dist是0，其他点的dist是 $+\infty$ ，满足 $\text{dist}[2] > \text{dist}[1] + c$ ，而 $+\infty$ 并不 $> +\infty + c$ ，所以第一次迭代结束就是不超过一条边走到i节点最短路的距离，依次类推，第二次迭代，只有3会被更新，因为只有1、2的dist不是 $+\infty$ ，第二次迭代就是不超过2条边走到i节点的最短距离。这就是为什么k次迭代最多是走了k条边，同时也是为什么一共只用迭代n-1次，因为n个点的有向图，如果能走到，原点到n号点的最短距离最多是n-1次，也就是1->2->...->n直线这种。

SPFA算法（队列优化的Bellman-Ford算法）

时间复杂度平均情况下 $O(m)$ ，最坏情况下 $O(nm)$ ，n表示点数，m表示边数

SPFA 核心思想

1. 只把距离被更新了的节点放进队列
2. 每次从队首取出节点，用它去松弛它的所有邻边
3. 如果某个邻点距离变小，且不在队列里，就入队
4. 重复直到队列为空

本质：只更新有变化的点，不瞎循环。

模板

```
int n;          // 总点数
int h[N], w[N], e[N], ne[N], idx;          // 邻接表存储所有边
int dist[N];    // 存储每个点到1号点的最短距离
bool st[N];     // 存储每个点是否在队列中

// 求1号点到n号点的最短路距离，如果从1号点无法走到n号点则返回-1
int spfa()
{
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;

    queue<int> q;
    q.push(1);
    st[1] = true;

    while (q.size())
    {
        auto t = q.front();
        q.pop();

        st[t] = false;

        for (int i = h[t]; i != -1; i = ne[i])
        {
            int j = e[i];
            if (dist[j] > dist[t] + w[i])
            {
                dist[j] = dist[t] + w[i];
                if (!st[j])    // 如果队列中已存在j，则不需要将j重复插入
                {
                    q.push(j);
                    st[j] = true;
                }
            }
        }
    }

    if (dist[n] == 0x3f3f3f3f) return -1;
    return dist[n];
}
```

应用

```
const int N = 1e6 + 10;

int n, m; // 节点数量和边数
int h[N], w[N], e[N], ne[N], idx; // 邻接矩阵存储图
int dist[N]; // 存储距离
bool st[N]; // 存储状态

void add(int a, int b, int c)
{
    e[idx] = b, w[idx] = c, ne[idx] = h[a], h[a] = idx ++ ;
}

int spfa()
{
    memset(dist, 0x3f, sizeof dist); // 距离初始化为无穷大
    dist[1] = 0; // 初始化1到1的距离为0
    queue<int> que; // 队列
    que.push(1); // 1入队

    while (que.size()) // 判断是否存在
    {
        int t = que.front();
        que.pop(); // 获取第一个并出队
        st[t] = false; // 第一个取消占用
        for (int i = h[t]; i != -1; i = ne[i]) // 遍历第一个可以到达的结点
        {
            int j = e[i];
            if (dist[j] > dist[t] + w[i]) // 1号点可到达的节点距离是否大于上次的距离距离加上当前的距离
            {
                dist[j] = dist[t] + w[i]; // 赋值给可到达的节点
                if (!st[j]) // 如果可到达的节点未被占用
                {
                    que.push(j); // 则入队
                    st[j] = true; // 占用
                }
            }
        }
    }

    return dist[n];
}

int main()
{
    scanf("%d%d", &n, &m);

    memset(h, -1, sizeof h);
    while (m -- )
    {
        int a, b, c;
        scanf("%d%d%d", &a, &b, &c);
        add(a, b, c);
    }
}
```

```

int t=spfa();
if(t==0x3f3f3f3f)cout<<"impossible"<<endl;
else printf("%d\n",t);

return 0;
}

```

应用：spfa判断图中是否存在负权

```

int n;          // 总点数
int h[N], w[N], e[N], ne[N], idx;          // 邻接表存储所有边
int dist[N], cnt[N];          // dist[x]存储1号点到x的最短距离，cnt[x]存储1到x的最短路中经过的点数
bool st[N];      // 存储每个点是否在队列中

// 如果存在负环，则返回true，否则返回false。
bool spfa()
{
    // 不需要初始化dist数组
    // 原理：如果某条最短路径上有n个点（除了自己），那么加上自己之后一共有n+1个点，由抽屉原理一定有两个点相同，所以存在环。

    queue<int> q;
    for (int i = 1; i <= n; i ++ )
    {
        q.push(i);
        st[i] = true;
    }

    while (q.size())
    {
        auto t = q.front();
        q.pop();

        st[t] = false;

        for (int i = h[t]; i != -1; i = ne[i])
        {
            int j = e[i];
            if (dist[j] > dist[t] + w[i])
            {
                dist[j] = dist[t] + w[i];
                cnt[j] = cnt[t] + 1;
                if (cnt[j] >= n) return true;          // 如果从1号点到x的最短路中包含至少n个点（不包括自己），则说明存在环
                if (!st[j])
                {
                    q.push(j);
                    st[j] = true;
                }
            }
        }
    }
}

```

```
    return false;
}
```

floyd算法

本质floyd算法是动态规划

时间复杂度 $O(n^3)$, n 表示点数

视频讲解: <https://www.bilibili.com/video/BV19k4y1Q7Gj/>

模板

初始化:

```
for (int i = 1; i <= n; i ++ )
    for (int j = 1; j <= n; j ++ )
        if (i == j) d[i][j] = 0;
        else d[i][j] = INF;

// 算法结束后, d[a][b]表示a到b的最短距离
void floyd()
{
    for (int k = 1; k <= n; k ++ )//k为中转节点
        for (int i = 1; i <= n; i ++ )
            for (int j = 1; j <= n; j ++ )
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

应用

```
using namespace std;

const int N = 210, INF = 1e9;

int n, m, Q;
int d[N][N];

void floyd()
{
    for (int k = 1; k <= n; k ++ )//k为中转节点
        for (int i = 1; i <= n; i ++ )
            for (int j = 1; j <= n; j ++ )
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}

int main()
{
    scanf("%d%d%d", &n, &m, &Q);

    for (int i = 1; i <= n; i ++ )
        for (int j = 1; j <= n; j ++ )
            if (i == j) d[i][j] = 0;
```

```

        else d[i][j] = INF;

    while (m -- )
    {
        int a, b, c;
        scanf("%d%d%d", &a, &b, &c);
        d[a][b] = min(d[a][b], c);
    }

    floyd();

    while (Q -- )
    {
        int a, b;
        scanf("%d%d", &a, &b);

        int t = d[a][b];
        if (t > INF / 2) puts("impossible");
        else printf("%d\n", t);
    }

    return 0;
}

```

最短路算法总结

最短路

单源最短路：给定V中的一个顶点，称为源。要计算从源到其他所有各顶点的最短路径长度。这里的长度就是指路上各边权之和。这个问题通常称为单源最短路径问题。

所有边权都是正数：

朴素Dijkstra算法 $O(n^2)$ 适合稠密图，贪心思想

堆优化版的Dijkstra算法 $O(m\log n)$ 适合稀疏图，贪心思想

存在负权边：

Bellman-ford $O(nm)$ ，动态规划思想

SPFA 一般： $O(m)$ ，最坏 $O(nm)$

多源汇最短路：任意两点最短路径被称为多源最短路径，即给定任意两个点，一个出发点，一个到达点，求这两个点的之间的最短路径，就是任意两点最短路径问题

Floyd算法 $O(n^3)$

Prim和Kruskal求最小生成树

最小生成树是一个连通加权无向图中所有顶点的一棵子树，其边的权重总和最小，且没有环。

最小生成树算法视频讲解：<https://www.bilibili.com/video/BV1wG411z79G/>

prim算法

朴素版

时间复杂度是 $O(n^2)$ ，n表示点数，m表示边数，适用场景点少、边极多的稠密图

```

int n;          // n表示点数
int g[N][N];    // 邻接矩阵，存储所有边
int dist[N];    // 存储其他点到当前最小生成树的距离
bool st[N];     // 存储每个点是否已经在生成树中

// 如果图不连通，则返回INF(值是0x3f3f3f3f)，否则返回最小生成树的树边权重之和
int prim()
{
    memset(dist, 0x3f, sizeof dist);

    int res = 0; //权重之和
    for (int i = 0; i < n; i ++ )
    {
        // 不属于集合&&距离集合最小的点
        int t = -1;
        for (int j = 1; j <= n; j ++ )
            if (!st[j] && (t == -1 || dist[t] > dist[j]))
                t = j;
        // 如果条件成立，则此图不连通，无法构造最小生成树，故fan。
        if (i && dist[t] == INF) return INF;
        // 把点t加到集合当中去，更新权值
        if (i) res += dist[t];
        st[t] = true;
        // 更新dist, t到集合最小的距离
        for (int j = 1; j <= n; j ++ ) dist[j] = min(dist[j], g[t][j]);
    }

    return res;
}

```

堆优化版

时间复杂度是 $O(m\log m)$ ，n表示点数，m表示边数，适用场景边少、点多的稀疏图

```

// 堆优化Prim求最小生成树总权值，不连通返回0x3f3f3f3f
int prim()
{
    // 初始所有点到生成树边权设为无穷大
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;
    // 小根堆，优先取出边权最小的点
    priority_queue<PII, vector<PII>, greater<PII>> heap;
    heap.push({0, 1});

    int res = 0;

    while (heap.size())
    {
        auto t = heap.top();
        heap.pop();

        int ver = t.second;    // 当前取出的点
    }
}

```

```

    int d = t.first;          // 该点接入生成树的边权

    // 该点已经在生成树里，冗余堆元素直接跳过
    if (st[ver]) continue;
    st[ver] = true;
    res += d;

    // 遍历ver所有相连的边，更新邻接点到生成树的最短边
    for (int i = h[ver]; i != -1; i = ne[i])
    {
        int j = e[i];
        if (dist[j] > w[i])
        {
            dist[j] = w[i];
            heap.push({dist[j], j});
        }
    }
}

// 检查是否所有点都被纳入生成树，存在孤立点则图不连通
for (int i = 1; i <= n; i++)
{
    if (dist[i] == 0x3f3f3f3f)
        return 0x3f3f3f3f;
}
return res;
}

```

应用

```

using namespace std;

// 堆内元素: <边权, 点编号>
typedef pair<int, int> PII;

const int N = 1e6 + 10;

int n, m;          // n个点, m条无向边
int h[N], w[N], e[N], ne[N], idx; // 邻接表
int dist[N];        // dist[i]: 点i距离最小生成树集合的最小边权
bool st[N];         // st[i]: 标记点i是否已经加入最小生成树

// 添加一条 a -> b 权值为 c 的单向边
void add(int a, int b, int c)
{
    e[idx] = b;
    w[idx] = c;
    ne[idx] = h[a];
    h[a] = idx ++;
}

// 堆优化Prim求最小生成树总权值, 不连通返回0x3f3f3f3f
int prim()

```

```

{
    // 初始所有点到生成树边权设为无穷大
    memset(dist, 0x3f, sizeof dist);
    dist[1] = 0;
    // 小根堆，优先取出边权最小的点
    priority_queue<PII, vector<PII>, greater<PII>> heap;
    heap.push({0, 1});

    int res = 0;

    while (heap.size())
    {
        auto t = heap.top();
        heap.pop();

        int ver = t.second;    // 当前取出的点
        int d = t.first;       // 该点接入生成树的边权

        // 该点已经在生成树里，冗余堆元素直接跳过
        if (st[ver]) continue;
        st[ver] = true;
        res += d;

        // 遍历ver所有相连的边，更新邻接点到生成树的最短边
        for (int i = h[ver]; i != -1; i = ne[i])
        {
            int j = e[i];
            if (dist[j] > w[i])
            {
                dist[j] = w[i];
                heap.push({dist[j], j});
            }
        }
    }

    // 检查是否所有点都被纳入生成树，存在孤立点则图不连通
    for (int i = 1; i <= n; i++)
    {
        if (dist[i] == 0x3f3f3f3f)
            return 0x3f3f3f3f;
    }
    return res;
}

int main()
{
    scanf("%d%d", &n, &m);
    // 邻接表头初始化为-1
    memset(h, -1, sizeof h);

    while (m--)
    {
        int a, b, c;

```

```

scanf("%d%d%d", &a, &b, &c);
add(a, b, c);
add(b, a, c); // 无向边正反各存一次
}

int ans = prim();
if (ans == 0x3f3f3f3f)
    cout << "impossible" << endl;
else
    cout << ans << endl;

return 0;
}

```

Kruskal算法

时间复杂度 $O(m\log m)$, n 表示点数, m 表示边数

```

int n, m;           // n是点数, m是边数
int p[N];           // 并查集的父亲节点数组

struct Edge         // 存储边
{
    int a, b, w;
    bool operator< (const Edge &w) const
    {
        return w < w.w;
    }
}edges[M];

int find(int x)      // 并查集核心操作
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}

int kruskal()
{
    sort(edges, edges + m);

    for (int i = 1; i <= n; i++) p[i] = i; // 初始化并查集

    int res = 0, cnt = 0;
    for (int i = 0; i < m; i++)
    {
        int a = edges[i].a, b = edges[i].b, w = edges[i].w;

        a = find(a), b = find(b);
        if (a != b) // 如果两个连通块不连通, 则将这两个连通块合并
        {
            p[a] = b;
            res += w;
        }
    }
}

```

```

        cnt ++ ;
    }
}
if (cnt < n - 1) return INF;
return res;
}

```

应用

```

#include <cstring>
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 100010, M = 200010, INF = 0x3f3f3f3f;

int n, m;
int p[N];

struct Edge
{
    int a, b, w;

    bool operator< (const Edge &w) const
    {
        return w < w.w;
    }
}edges[M];

int find(int x)
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}

int kruskal()
{
    sort(edges, edges + m);

    for (int i = 1; i <= n; i ++ ) p[i] = i;    // 初始化并查集

    int res = 0, cnt = 0;
    for (int i = 0; i < m; i ++ )
    {
        int a = edges[i].a, b = edges[i].b, w = edges[i].w;

        a = find(a), b = find(b);
        if (a != b)
        {
            p[a] = b;
            res += w;
            cnt ++ ;
        }
    }
}

```

```

    }
}

if (cnt < n - 1) return INF;
return res;
}

int main()
{
    scanf("%d%d", &n, &m);

    for (int i = 0; i < m; i ++ )
    {
        int a, b, w;
        scanf("%d%d%d", &a, &b, &w);
        edges[i] = {a, b, w};
    }

    int t = kruskal();

    if (t == INF) puts("impossible");
    else printf("%d\n", t);

    return 0;
}

```

Prim和Kruskal算法讲解

最小生成树算法视频讲解: <https://www.bilibili.com/video/BV1wG411z79G/>

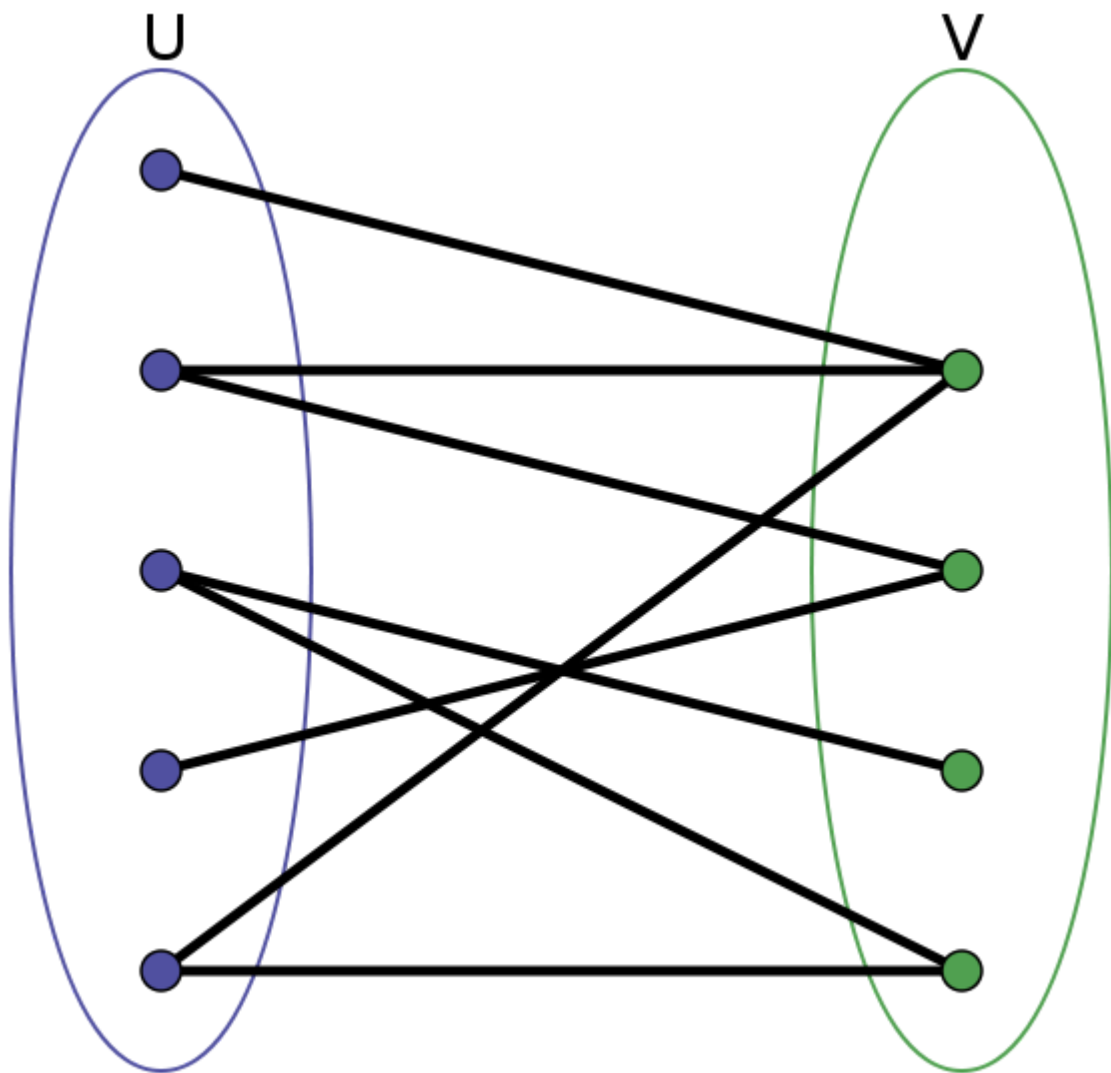
染色法判别二分图

什么叫二分图

有两顶点集且图中每条边的的两个顶点分别位于两个顶点集中，每个顶点集中没有边直接相连接！

说人话的定义：图中点通过移动能分成左右两部分，左侧的点只和右侧的点相连，右侧的点只和左侧的点相连。

下图就是个二分图：



时间复杂度是 $O(n + m)$, n 表示点数, m 表示边数

```
int n;           // n表示点数
int h[N], e[M], ne[M], idx; // 邻接表存储图
int color[N];    // 表示每个点的颜色, -1表示未染色, 0表示白色, 1表示黑色

// 参数: u表示当前节点, c表示当前点的颜色
bool dfs(int u, int c)
{
    color[u] = c;
    for (int i = h[u]; i != -1; i = ne[i])
    {
        int j = e[i];
        if (color[j] == -1)
        {
            if (!dfs(j, !c)) return false;
        }
        else if (color[j] == c) return false;
    }
}
```

```

        return true;
    }

    bool check()
    {
        memset(color, -1, sizeof color);
        bool flag = true;
        for (int i = 1; i <= n; i ++ )
            if (color[i] == -1)
                if (!dfs(i, 0))
                {
                    flag = false;
                    break;
                }
        return flag;
    }
}

```

应用

```

using namespace std;

const int N = 100010, M = 200010; // 由于是无向图，顶点数最大是N，那么边数M最大是顶点数的2倍

int n, m;
int h[N], e[M], ne[M], idx;
int color[N];

void add(int a, int b)
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
}

bool dfs(int u, int c)
{
    color[u] = c;

    for (int i = h[u]; i != -1; i = ne[i])
    {
        int j = e[i];
        if (!color[j])
        {
            if (!dfs(j, 3 - c)) return false;
        }
        else if (color[j] == c) return false;
    }

    return true;
}

int main()
{
    scanf("%d%d", &n, &m);
}

```

```

memset(h, -1, sizeof h);

while (m -- )
{
    int a, b;
    scanf("%d%d", &a, &b);
    add(a, b), add(b, a); // 无向图, a->b, b->a
}

bool flag = true;
for (int i = 1; i <= n; i ++ )
    if (!color[i])
    {
        if (!dfs(i, 1))
        {
            flag = false;
            break;
        }
    }

if (flag) puts("Yes");
else puts("No");

return 0;
}

```

匈牙利算法

要了解匈牙利算法必须先理解下面的概念：

匹配：在图论中，一个「匹配」是一个边的集合，其中任意两条边都没有公共顶点。

最大匹配：一个图所有匹配中，所含匹配边数最多的匹配，称为这个图的最大匹配。

下面是一些补充概念：

完美匹配：如果一个图的某个匹配中，所有的顶点都是匹配点，那么它就是一个完美匹配。

交替路：从一个未匹配点出发，依次经过非匹配边、匹配边、非匹配边...形成的路径叫交替路。

增广路：从一个未匹配点出发，走交替路，如果途径另一个未匹配点（出发的点不算），则这条交替路称为增广路（augmenting path）。

时间复杂度 $O(nm)$ ， n 表示点数， m 表示边数

```

//遍历自己喜欢的女孩int n1, n2; // n1表示第一个集合中的点数，n2表示第二个集合中的点数
int h[N], e[M], ne[M], idx; // 邻接表存储所有边，匈牙利算法中只会用到从第一个集合指向第二个集合的边，所以这里只用存一个方向的边
int match[N]; // 存储第二个集合中的每个点当前匹配的第一个集合中的点是哪个
bool st[N]; // 表示第二个集合中的每个点是否已经被遍历过

void add(int a, int b)
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
}

```

```

}

bool find(int x)
{
    //遍历自己喜欢的女孩
    for (int i = h[x]; i != -1; i = ne[i])
    {
        int j = e[i];
        if (!st[j])//如果在这一轮模拟匹配中,这个女孩尚未被预定
        {
            st[j] = true;//那x就预定这个女孩了
            //如果女孩j没有男朋友,或者她原来的男朋友能够预定其它喜欢的女孩。配对成功
            if (match[j] == 0 || find(match[j]))
            {
                match[j] = x;
                return true;
            }
        }
    }
    //自己中意的全部都被预定了。配对失败。
    return false;
}

// 求最大匹配数,依次枚举第一个集合中的每个点能否匹配第二个集合中的点
int res = 0;
for (int i = 1; i <= n1; i ++ )
{
    memset(st, false, sizeof st);
    if (find(i)) res ++ ;
}

```

应用：二分图的最大匹配

相关题解：[AcWing 861. 二分图的最大匹配----图解 - AcWing]: <https://www.acwing.com/solution/content/179030/>

```

using namespace std;

const int N = 510, M = 100010;

int n1, n2, m;
int h[N], e[M], ne[M], idx;
int match[N];
bool st[N];

void add(int a, int b)
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
}

bool find(int x)
{

```

```

    // 和各个点尝试能否匹配
    for (int i = h[x]; i != -1; i = ne[i])
    {
        int j = e[i];
        if (!st[j])//打标记
        {
            st[j] = true;
            // 当前尝试点没有被匹配或者和当前尝试点匹配的那个点可以换另一个匹配
            if (match[j] == 0 || find(match[j]))
            {
                // 和当前尝试点匹配在一起
                match[j] = x;
                return true;
            }
        }
    }
    return false;
}

int main()
{
    scanf("%d%d", &n1, &n2, &m);

    memset(h, -1, sizeof h);
    // 保存图，因为只从一边找另一边，所以该无向图只需要存储一个方向
    while (m -- )
    {
        int a, b;
        scanf("%d%d", &a, &b);
        add(a, b);
    }

    int res = 0;
    //为各个点找匹配
    for (int i = 1; i <= n1; i ++ )
    {
        memset(st, false, sizeof st);
        //找到匹配
        if (find(i)) res ++ ;
    }

    printf("%d\n", res);

    return 0;
}

```

四、数学知识

算法的数学知识定理证明可以在这里查阅：[数学部分简介 - OI Wiki (oi-wiki.org)]: <https://oi-wiki.org/math/>

试除法判定质数

```
bool is_prime(int x)
{
    if (x < 2) return false;
    for (int i = 2; i <= x / i; i ++ )
        if (x % i == 0)
            return false;
    return true;
}
```

试除法分解质因数

```
void divide(int x)
{
    for (int i = 2; i <= x / i; i ++ )
        if (x % i == 0) // i 一定是质数
        {
            int s = 0;
            while (x % i == 0) x /= i, s ++ ;
            cout << i << ' ' << s << endl;
        }
    if (x > 1) cout << x << ' ' << 1 << endl;
    cout << endl;
}
```

埃氏筛法求质数

```
int primes[N], cnt;    // primes[] 存储所有素数
bool st[N];            // st[x] 存储x是否被筛掉

void get_primes(int n)
{
    for (int i = 2; i <= n; i ++ )
    {
        if (st[i]) continue;
        primes[cnt ++ ] = i;
        for (int j = i + i; j <= n; j += i)
            st[j] = true;
    }
}
```

线性筛法求质数

```
int primes[N], cnt;    // primes[] 存储所有素数
bool st[N];            // st[x] 存储x是否被筛掉

void get_primes(int n)
{

```

```

for (int i = 2; i <= n; i ++ )
{
    if (!st[i]) primes[cnt ++ ] = i;
    for (int j = 0; primes[j] <= n / i; j ++ )
    {
        st[primes[j] * i] = true;
        if (i % primes[j] == 0) break;
    }
}
}

```

试除法求所有约数

```

vector<int> get_divisors(int x)
{
    vector<int> res;
    for (int i = 1; i <= x / i; i ++ )
        if (x % i == 0)
        {
            res.push_back(i);
            if (i != x / i) res.push_back(x / i);
        }
    sort(res.begin(), res.end());
    return res;
}

```

约数个数

约数个数定理和约数和定理公式推导: <https://www.bilibili.com/video/BV13R4y1o777>

约数个数定理推导: <https://www.bilibili.com/video/BV1NY41187GM>

$$360 = 2^3 \times 3^2 \times 5^1$$

$$\text{约数个数} = (3+1) \times (2+1) \times (1+1)$$

```

using namespace std;
typedef long long LL;
const int N = 110, mod = 1e9 + 7;
int main()
{
    int n;
    cin >> n;
    unordered_map<int, int> primes;
    while (n -- )
    {
        int x;
        cin >> x;
    }
}

```

```

        for (int i = 2; i <= x / i; i ++ )
            while (x % i == 0)
            {
                x /= i;
                primes[i] ++ ;
            }
        if (x > 1) primes[x] ++ ;
    }
    LL res = 1;
    for (auto p : primes) res = res * (p.second + 1) % mod;
    cout << res << endl;
    return 0;
}

```

约数之和

约数个数定理和约数和定理公式推导: <https://www.bilibili.com/video/BV13R4y1o777>

$$\because 360 = 2^3 \times 3^2 \times 5^1$$

$$\therefore \text{约数个数} = (3+1) \times (2+1) \times (1+1)$$

$$\text{约数之和} = (2^0 + 2^1 + 2^2 + 2^3) \times (3^0 + 3^1 + 3^2) \times (5^0 + 5^1)$$

```

using namespace std;
typedef long long LL;
const int N = 110, mod = 1e9 + 7;
int main()
{
    int n;
    cin >> n;
    unordered_map<int, int> primes;
    while (n -- )
    {
        int x;
        cin >> x;
        for (int i = 2; i <= x / i; i ++ )
            while (x % i == 0)
            {
                x /= i;
                primes[i] ++ ;
            }
        if (x > 1) primes[x] ++ ;
    }
    LL res = 1;
    for (auto p : primes)
    {
        LL a = p.first, b = p.second;
        LL t = 1;

```

```

while (b -- ) t = (t * a + 1) % mod; //遍历b次后得到t=p^b+p^(b-1)+...+p+1
res = res * t % mod;
}
cout << res << endl;
return 0;
}

```

代码第26行解释:

while(b--) t=a*t+1;

当b=3时, 经过3次迭代如下

t=1

t=a+1

t=a*(a+1)+1=a^2+a+1

t=a*(a^2+a+1)+1=a^3+a^2+a+1

约数个数、约数之和的数学公式

$$N = \prod_{i=1}^k p_i^{a_i} = p_1^{a_1} \cdot p_2^{a_2} \cdots p_k^{a_k}$$

$$\text{约数个数: } \prod_{i=1}^k (a_i + 1) = (a_1 + 1)(a_2 + 1) \cdots (a_k + 1)$$

$$\text{约数之和: } \prod_{i=1}^k \sum_{j=0}^{a_i} p_i^j = \prod_{i=1}^k (p_i^0 + p_i^1 + \cdots + p_i^{a_i})$$

$$= (p_1^0 + p_1^1 + \cdots + p_1^{a_1})(p_2^0 + p_2^1 + \cdots + p_2^{a_2}) \cdots (p_k^0 + p_k^1 + \cdots + p_k^{a_k})$$

欧几里得算法(求最大公约数)

```

int gcd(int a, int b)
{
    return b ? gcd(b, a % b) : a;
}

```

最小公倍数

```
int lcm(int a, int b)
{
    return abs(a * b) / gcd(a, b);
}
```

求欧拉函数

前置知识

互质：互质是公因数只有1的两个整数，叫做互质整数。

欧拉函数定义

$1 \sim N - 1$ 中与 N 互质的数的个数被称为欧拉函数，记为 $\phi(N)$ 。

若在算数基本定理中， $N = p_1^{a_1} p_2^{a_2} \dots p_m^{a_m}$ ，则：

$$\phi(N) = N \cdot \frac{p_1-1}{p_1} \cdot \frac{p_2-1}{p_2} \cdot \dots \cdot \frac{p_m-1}{p_m}$$

欧拉函数推导

首先我们要知道 $1, 2, 3 \dots N - 1, N$ 与 N 互质的个数是 $1 \sim N$ 数列去除 N 的质因子的倍数。

例如 $N = 10$ ，即 $1, 2, 3, 4, 5, 6, 7, 8, 9, 10$ 去除 N 的质因子的倍数，则~~1, 2, 3, 4, 5, 6, 7, 8, 9, 10~~。

显然， $1, 3, 7, 9$ 与 10 互质。

由上方结论使用容斥原理进行数学推导如下：

$$\because N = p_1^{a_1} p_2^{a_2} \dots p_m^{a_m}$$

①.从 $1 \sim n$ 中去掉 $p_1, p_2, \dots, p_k$ 的所有倍数的个数，即

$$n \leftarrow n - \frac{n}{p_1} - \frac{n}{p_2} - \dots - \frac{n}{p_k}$$

②.由容斥原理， $p_i \cdot p_j$ 的倍数被①减了两次，所以加上所有 $p_i \cdot p_j$ 的倍数的个数（其中 p_i, p_j 是 $p_1 \sim p_k$ 的组合），即

$$n \leftarrow n + \frac{n}{p_1 \cdot p_2} + \frac{n}{p_1 \cdot p_3} + \dots + \frac{n}{p_{k-1} \cdot p_k}$$

③.减去所有 $p_i \cdot p_j \cdot p_k$ 的倍数个数，即

$$n \leftarrow n - \frac{n}{p_1 \cdot p_2 \cdot p_3} - \frac{n}{p_1 \cdot p_2 \cdot p_4} - \dots - \frac{n}{p_{k-2} \cdot p_{k-1} \cdot p_k}$$

④.同理，加上所有 $p_i \cdot p_j \cdot p_k \cdot p_l$ 的倍数个数，即

$$n \leftarrow n + \frac{n}{p_1 \cdot p_2 \cdot p_3 \cdot p_4} + \frac{n}{p_1 \cdot p_2 \cdot p_3 \cdot p_5} + \dots + \frac{n}{p_{k-3} \cdot p_{k-2} \cdot p_{k-1} \cdot p_k}$$

$$\vdots$$

因此，

$$\begin{aligned}\phi(n) = & n - \frac{n}{p_1} - \frac{n}{p_2} - \dots - \frac{n}{p_k} \\ & + \frac{n}{p_1 \cdot p_2} + \frac{n}{p_1 \cdot p_3} + \dots + \frac{n}{p_{k-1} \cdot p_k} \\ & - \frac{n}{p_1 \cdot p_2 \cdot p_3} - \frac{n}{p_1 \cdot p_2 \cdot p_4} - \dots - \frac{n}{p_{k-2} \cdot p_{k-1} \cdot p_k} \\ & + \frac{n}{p_1 \cdot p_2 \cdot p_3 \cdot p_4} + \frac{n}{p_1 \cdot p_2 \cdot p_3 \cdot p_5} + \dots + \frac{n}{p_{k-3} \cdot p_{k-2} \cdot p_{k-1} \cdot p_k} \\ & - \dots\end{aligned}$$

也就是n减去奇数个质因子的倍数个数，加上偶数个质因子的倍数个数，循环往复。

将上式等价变形，得到

$$\phi(n) = n \cdot \left(1 - \frac{1}{p_1}\right) \cdot \left(1 - \frac{1}{p_2}\right) \dots \cdot \left(1 - \frac{1}{p_k}\right)$$

证必。

代码模板

```
int phi(int x)
{
    int res = x;
    for (int i = 2; i <= x / i; i++)
        if (x % i == 0)
        {
            res = res / i * (i - 1);
            while (x % i == 0) x /= i;
        }
    if (x > 1) res = res / x * (x - 1);

    return res;
}
```

线性筛法求欧拉函数

```
int primes[N], cnt;    // primes[] 存储所有素数
int euler[N];          // 存储每个数的欧拉函数
bool st[N];            // st[x] 存储x是否被筛掉

void get_eulers(int n) // 线性筛法求1~n的欧拉函数
{
    euler[1] = 1;
    for (int i = 2; i <= n; i++)
    {
        if (!st[i])
        {
            primes[cnt++] = i;
            euler[i] = i - 1;
        }
        for (int j = 0; primes[j] <= n / i; j++)
        {
            int t = primes[j] * i;
            st[t] = true;
            if (i % primes[j] == 0)
            {
                euler[t] = euler[i] * primes[j];
                break;
            }
            euler[t] = euler[i] * (primes[j] - 1);
        }
    }
}
```

```

        if (i % primes[j] == 0)
        {
            euler[t] = euler[i] * primes[j];
            break;
        }
        euler[t] = euler[i] * (primes[j] - 1);
    }
}
}

```

快速幂

快速幂公式证明: [快速幂 - OI Wiki (oi-wiki.org)]: <https://oi-wiki.org/math/binary-exponentiation/>

```

// 求  $m^k \bmod p$ , 时间复杂度  $O(\log k)$ 。
// m为底数, k为幂
int qmi(int m, int k, int p)
{
    int res = 1 % p, t = m;
    while (k)
    {
        if (k&1) res = res * t % p;
        t = t * t % p;
        k >>= 1;
    }
    return res;
}

```

扩展欧几里得算法

扩展欧几里得算法讲解: <https://www.bilibili.com/video/BV1KU4y1a7E2/>

优秀题解: <https://www.acwing.com/solution/content/1393>

优秀博客: <https://blog.csdn.net/mango114514/article/details/121048335>

x的第一个正解就是 $(x\%k+k)\%k$

其中, $k=b/\gcd(a,b)$

```

// 求x, y, 使得 $ax + by = \gcd(a, b)$ 
int exgcd(int a, int b, int &x, int &y)
{
    if (!b)
    {
        x = 1, y = 0;
        return a;
    }
    int d = exgcd(b, a % b, y, x);
    y -= (a/b) * x;
    return d;
}

```

中国剩余定理

中国剩余定理讲解: <https://www.bilibili.com/video/BV1AN4y1N7Su/>

《孙子算经》

有物不知其数，三三数之剩二，五五数之剩三，七七数之剩二。问物几何？

求解线性同余方程组

$$\begin{cases} x \equiv r_1 \pmod{m_1} \\ x \equiv r_2 \pmod{m_2} \\ \vdots \\ x \equiv r_n \pmod{m_n} \end{cases}$$

其中模数 $m_1, m_2, \dots, m_n$ 为两两互质的整数，求 x 的最小非负整数解。

中国剩余定理 (Chinese Remainder Theorem, CRT)

1. 计算所有模数的积 M
2. 计算第 i 个方程的 $c_i = \frac{M}{m_i}$
3. 计算 c_i 在模 m_i 意义下的逆元 c_i^{-1}
4. $x = \sum_{i=1}^n r_i c_i c_i^{-1} \pmod{M}$

例
$$\begin{cases} x \equiv 2 \pmod{3} \\ x \equiv 3 \pmod{4} \\ x \equiv 1 \pmod{5} \end{cases}$$

1. $M = 3 \times 4 \times 5 = 60$
2. $c_1 = 20, c_1^{-1} = 2$, 因 $20x \equiv 1 \pmod{3}$
 $c_2 = 15, c_2^{-1} = 3$, 因 $15x \equiv 1 \pmod{4}$
 $c_3 = 12, c_3^{-1} = 3$, 因 $12x \equiv 1 \pmod{5}$
3. $x = (2 \times 20 \times 2 + 3 \times 15 \times 3 + 1 \times 12 \times 3) \% 60$
 $= 251 \% 60 = 11$

证明:

先证明 $\sum_{i=1}^n r_i c_i c_i^{-1}$ 满足每个 $x \equiv r_i \pmod{m_i}$ 。

当 $i \neq j$ 时, $c_j \equiv 0 \pmod{m_i}$, 则 $c_j c_j^{-1} \equiv 0 \pmod{m_i}$

当 $i = j$ 时, $c_i \not\equiv 0 \pmod{m_i}$, 则 $c_i c_i^{-1} \equiv 1 \pmod{m_i}$

故 $x \equiv \sum_{j=1}^n r_j c_j c_j^{-1} \pmod{m_i}$

$$\equiv r_i c_i c_i^{-1} \pmod{m_i}$$

$$\equiv r_i \pmod{m_i}$$

而 $\sum_{i=1}^n r_i c_i c_i^{-1} \pmod{M}$ 对 m_i 来说, 只是减去了 m_i 的若干倍, 不影响余数 r_i 。证毕

CSG1 98E 页

```
LL exgcd(LL a, LL b, LL &x, LL &y){
    if(b==0){
        x=1, y=0;
        return a;
    }
    LL d=exgcd(b, a%b, y, x);
    y -= (a/b) * x;
    return d;
}
LL CRT(LL m[], LL r[]){
    LL m=1, ans=0;
    for(int i=1; i<=n; i++) m*=m[i];
    for(int i=1; i<=n; i++){
        LL c=m/m[i], x, y;
        exgcd(c, m[i], x, y);
        ans=(ans+r[i]*c*x%M)%M;
    }
    return (ans%M+M)%M;
}
```

扩展中国剩余定理

扩展中国剩余定理讲解: <https://www.bilibili.com/video/BV1Ut4y1F7HG/>

问题

求解线性同余方程组

$$\begin{cases} x \equiv r_1 \pmod{m_1} \\ x \equiv r_2 \pmod{m_2} \\ \dots \\ x \equiv r_n \pmod{m_n} \end{cases}$$

其中 $m_1, m_2, \dots, m_n$ 为不一定两两互质的整数，求 x 的最小非负整数解。

中国剩余定理 (CRT) 已不可行

其构造解 $x = \sum_{i=1}^n r_i c_i c_i^{-1} \pmod{M}$

其中 $c_i x \equiv 1 \pmod{m_i}$ ，即 $c_i x + m_i y = 1 = \gcd(c_i, m_i)$

根据裴蜀定理， c_i, m_i 应该互质， $c_i = \frac{m_1 \times \dots \times m_n}{m_i}$

如果 c_i, m_i 不互质，则 c_i^{-1} 不存在，算法失效

扩展中国剩余定理 (EXCRT)

前两个方程： $x \equiv r_1 \pmod{m_1}, x \equiv r_2 \pmod{m_2}$

转化为不定方程： $x = m_1 p + r_1 = m_2 q + r_2$

则 $m_1 p - m_2 q = r_2 - r_1$

由裴蜀定理，

当 $\gcd(m_1, m_2) \nmid (r_2 - r_1)$ 时，无解

当 $\gcd(m_1, m_2) \mid (r_2 - r_1)$ 时，有解

由扩欧算法，

得特解 $p = p * \frac{r_2 - r_1}{\gcd}, q = q * \frac{r_2 - r_1}{\gcd}$

其通解 $P = p + \frac{m_2}{\gcd} * k, Q = q - \frac{m_1}{\gcd} * k$

所以 $x = m_1 P + r_1 = \frac{m_1 m_2}{\gcd} * k + m_1 p + r_1$

前两个方程等价合并为一个方程 $x \equiv r \pmod{m}$

其中 $r = m_1 p + r_1, m = \text{lcm}(m_1, m_2)$

所以 n 个同余方程只要合并 $n - 1$ 次，即可求解

CCO 08 页

```
LL exgcd(LL a, LL b, LL &x, LL &y){
    if(b==0){
        x=1, y=0;
        return a;
    }
    LL d=exgcd(b, a%b, y, x);
    y -= (a/b) * x;
    return d;
}
LL EXCRT(LL m[], LL r[]){
    LL m1, m2, r1, r2, p, q;
    m1=m[1], r1=r[1];
    for(int i=2; i<=n; i++){
        m2=m[i], r2=r[i];
        LL d = exgcd(m1, m2, p, q);
        if((r2-r1)%d){
            return -1;
        }
        p=p*(r2-r1)/d; //特解
        p=(p%(m2/d)+m2/d)%(m2/d);
        r1=m1*p+r1;
        m1=m1*m2/d;
    }
    return (r1%m1+m1)%m1;
}
```

高斯消元法

高斯消元 $O(n^3)$

求解例如下面方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \vdots \qquad \qquad \qquad \vdots \qquad \qquad \qquad \vdots \qquad \qquad \qquad \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = b_n \end{cases}$$

高斯消元讲解: <https://www.bilibili.com/video/BV1Kd4y127vZ/>

模板

```
// a[N][N]是增广矩阵
int gauss()
{
    int c, r;
    for (c = 0, r = 0; c < n; c++)
    {
        int t = r;
        for (int i = r; i < n; i++) // 找到绝对值最大的行
            if (fabs(a[i][c]) > fabs(a[t][c]))
                t = i;

        if (fabs(a[t][c]) < eps) continue;

        for (int i = c; i <= n; i++) swap(a[t][i], a[r][i]); // 将绝对值最大的行换到最顶端

        for (int i = n; i >= c; i--) a[r][i] /= a[r][c]; // 将当前行的首位变成1
        for (int i = r + 1; i < n; i++) // 用当前行将下面所有的列消成0
            if (fabs(a[i][c]) > eps)
                for (int j = n; j >= c; j--)
                    a[i][j] -= a[r][j] * a[i][c];

        r++;
    }

    if (r < n)
    {
        for (int i = r; i < n; i++)
            if (fabs(a[i][n]) > eps)
                return 2; // 无解
        return 1; // 有无穷多组解
    }

    for (int i = n - 1; i >= 0; i--)
        for (int j = i + 1; j < n; j++)
            a[i][n] -= a[i][j] * a[j][n];

    return 0; // 有唯一解
}
```

应用

```
using namespace std;

const int N = 110;
```

```

const double eps = 1e-6;

int n;
double a[N][N];

int gauss()
{
    int c, r; // c 代表 列 col , r 代表 行 row
    for (c = 0, r = 0; c < n; c++)
    {
        int t = r; // 先找到当前这一列, 绝对值最大的一个数字所在的行号
        for (int i = r; i < n; i++)
            if (fabs(a[i][c]) > fabs(a[t][c]))
                t = i;

        if (fabs(a[t][c]) < eps) continue; // 如果当前这一列的最大数都是 0, 那么所有数都是 0, 就没必要去算了, 因为它的约束方程, 可能在上面几行

        for (int i = c; i < n + 1; i++) swap(a[t][i], a[r][i]); // 把当前这一行, 换到最上面 (不是第一行, 是第 r 行) 去
        for (int i = n; i >= c; i--) a[r][i] /= a[r][c]; // 把当前这一行的第一个数, 变成 1, 方程两边同时除以 第一个数, 必须要到着算, 不然第一个数直接变1, 系数就被篡改, 后面的数字没法算
        for (int i = r + 1; i < n; i++) // 把当前列下面的所有数, 全部消成 0
            if (fabs(a[i][c]) > eps) // 如果非0 再操作, 已经是 0就没必要操作了
                for (int j = n; j >= c; j--) // 从后往前, 当前行的每个数字, 都减去对应列 * 行首非0 的数字, 这样就能保证第一个数字是 a[i][0] -= 1*a[i][0];
                    a[i][j] -= a[r][j] * a[i][c];

        r++; // 这一行的工作做完, 换下一行
    }

    if (r < n) // 说明剩下方程的个数是小于 n 的, 说明不是唯一解, 判断是无解还是无穷多解
    { // 因为已经是阶梯型, 所以 r ~ n-1 的值应该都为 0
        for (int i = r; i < n; i++) //
            if (fabs(a[i][n]) > eps) // a[i][n] 代表 b_i ,即 左边=0, 右边=b_i, 0 != b_i, 所以无解。

                return 2;
        return 1; // 否则, 0 = 0, 就是 r ~ n-1 的方程都是多余方程
    }

    // 唯一解 ↓, 从下往上回代, 得到方程的解
    for (int i = n - 1; i >= 0; i--)
        for (int j = i + 1; j < n; j++)
            a[i][n] -= a[j][n] * a[i][j]; // 因为只要得到解, 所以只用对 b_i 进行操作, 中间的值, 可以不用操作, 因为不用输出

    return 0;
}

int main()
{
    cin >> n;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n + 1; j++)

```

```
        cin >> a[i][j];

    int t = gauss();

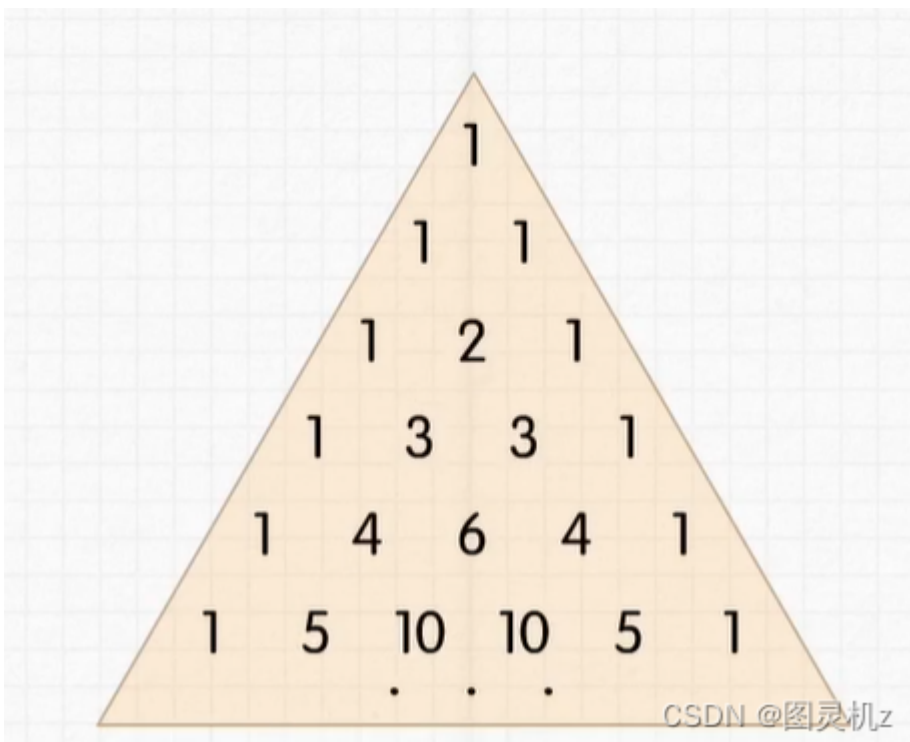
    if (t == 0)
    {
        for (int i = 0; i < n; i ++ ) printf("%.2lf\n", a[i][n]);
    }
    else if (t == 1) puts("Infinite group solutions");
    else puts("No solution");
    return 0;
}
```

求组合数

递推法求组合数

排列组合详细讲解1: <https://www.bilibili.com/video/BV1e7411J7SC/>

排列组合详细讲解2: https://blog.csdn.net/qg_39757593/article/details/129804636



$$\begin{array}{c}
 C_0^0 \\
 C_1^0 \quad C_1^1 \\
 C_2^0 \quad C_2^1 \quad C_2^2 \\
 C_3^0 \quad C_3^1 \quad C_3^2 \quad C_3^3 \\
 C_4^0 \quad C_4^1 \quad C_4^2 \quad C_4^3 \quad C_4^4
 \end{array}$$

1. 左右两侧斜线都是1, $C_n^0 = C_n^n = 1$
2. 其他数等于其左上角和右上角两数之和 $C_n^{m-1} + C_n^m = C_{n+1}^m$

```
// c[a][b] 表示从a个苹果中选b个的方案数
int c[N][N];
for (int i = 0; i < N; i++)
    for (int j = 0; j <= i; j++)
        if (!j) c[i][j] = 1;
        else c[i][j] = (c[i-1][j] + c[i-1][j-1]) % mod;
//本质上杨辉三角
```

通过预处理逆元的方式求组合数

模板

```
// 首先预处理出所有阶乘取模的余数fact[N]，以及所有阶乘取模的逆元infact[N]
// 如果取模的数是质数，可以用费马小定理求逆元
int qmi(int a, int k, int p)    // 快速幂模板
{
    int res = 1;
    while (k)
    {
        if (k & 1) res = (LL)res * a % p;
        a = (LL)a * a % p;
        k >>= 1;
    }
    return res;
}

// 预处理阶乘的余数和阶乘逆元的余数
fact[0] = infact[0] = 1;
for (int i = 1; i < N; i++)
{
    fact[i] = (LL)fact[i-1] * i % mod;
```

```
    infact[i] = (LL)infact[i - 1] * qmi(i, mod - 2, mod) % mod;
}
```

应用

```
using namespace std;

typedef long long LL;

const int N = 100010, mod=1e9+7; //1e9+7是质数所以与[1,1e9+7)中的数互质

int fact[N], infact[N];

int qmi(int a, int k, int p){
    int res=1;
    while(k){
        if(k&1) res=(LL)res*a%p;
        a=(LL)a*a%p;
        k>>=1;
    }
    return res;
}

int main()
{
    fact[0]=infact[0]=1;
    for (int i = 1; i <= N; i ++ ){
        fact[i]=(LL)fact[i-1]*i%mod;
        infact[i]=(LL)infact[i-1]*qmi(i, mod-2, mod)%mod;
    }

    int n;
    scanf("%d",&n);
    while (n -- ){
        int a,b;
        scanf("%d%d", &a, &b);
        printf("%d\n", (LL)fact[a]*infact[b]%mod*infact[a-b]%mod);
    }
    return 0;
}
```

Lucas定理求组合数

Lucas定理证明: https://blog.csdn.net/Qiuker_jl/article/details/109528164

模板

```
// 若p是质数，则对于任意整数  $1 \leq m \leq n$ ，有：
//  $C(n, m) = C(n \% p, m \% p) * C(n / p, m / p) \pmod p$ 

int qmi(int a, int k, int p) // 快速幂模板
{
```

```

int res = 1 % p;
while (k)
{
    if (k & 1) res = (LL)res * a % p;
    a = (LL)a * a % p;
    k >>= 1;
}
return res;
}

int C(int a, int b, int p) // 通过定理求组合数C(a, b)
{
    if (a < b) return 0;

    LL x = 1, y = 1; // x是分子, y是分母
    for (int i = a, j = 1; j <= b; i --, j ++ )
    {
        x = (LL)x * i % p;
        y = (LL) y * j % p;
    }

    return x * (LL)qmi(y, p - 2, p) % p;
}

int lucas(LL a, LL b, int p)
{
    if (a < p && b < p) return C(a, b, p);
    return (LL)C(a % p, b % p, p) * lucas(a / p, b / p, p) % p;
}

```

应用

```

using namespace std;

typedef long long LL;

int qmi(int a,int k,int p)
{
    int res = 1;
    while(k)
    {
        if(k&1)res = (LL)res*a%p;
        a = (LL)a*a%p;
        k>>=1;
    }
    return res;
}

int C(int a,int b,int p)//自变量类型int
{
    if(b>a)return 0;//漏了边界条件
    int res = 1;
    // a!/(b!(a-b)!) = (a-b+1)*...*a / b! 分子有b项

```

```

for(int i=1,j=a;i<=b;i++,j--)//i<=b而不是<
{
    res = (LL)res*j%p;
    res = (LL)res*qmi(i,p-2,p)%p;
}
return res;
}
//对公式敲
int lucas(LL a,LL b,int p)
{
    if(a<p && b<p)return C(a,b,p);//lucas递归终点是C_{bk}^{ak}
    return (LL)C(a%p,b%p,p)*lucas(a/p,b/p,p)%p;//a%p后肯定是<p的,所以可以用C(),但a/p后不一定<p 所以用lucas继续递归
}

int main()
{
    int n;
    cin >> n;
    while(n-->0)
    {
        LL a,b;
        int p;
        cin >> a >> b >> p;
        cout << lucas(a,b,p) << endl;
    }
    return 0;
}

```

分解质因数法求组合数

模板

当我们需要求出组合数的真实值，而非对某个数的余数时，分解质因数的方式比较好用：

1. 筛法求出范围内的所有质数
2. 通过 $C(a, b) = \frac{a!}{b! \cdot (a-b)!}$ 这个公式求出每个质因子的次数。 $n!$ 中 p 的次数是 $n/p + n/p^2 + n/p^3 + \dots$
3. 用高精度乘法将所有质因子相乘

```

int primes[N], cnt;    // 存储所有质数
int sum[N];           // 存储每个质数的次数
bool st[N];           // 存储每个数是否已被筛掉

void get_primes(int n)    // 线性筛法求素数
{
    for (int i = 2; i <= n; i++)
    {
        if (!st[i]) primes[cnt++] = i;
        for (int j = 0; primes[j] <= n / i; j++)
        {
            st[primes[j] * i] = true;

```

```

        if (i % primes[j] == 0) break;
    }
}

int get(int n, int p)    // 求n! 中的次数
{
    int res = 0;
    while (n)
    {
        res += n / p;
        n /= p;
    }
    return res;
}

vector<int> mul(vector<int> a, int b)    // 高精度乘低精度模板
{
    vector<int> c;
    int t = 0;
    for (int i = 0; i < a.size(); i++)
    {
        t += a[i] * b;
        c.push_back(t % 10);
        t /= 10;
    }

    while (t)
    {
        c.push_back(t % 10);
        t /= 10;
    }

    return c;
}

get_primes(a);    // 预处理范围内的所有质数

for (int i = 0; i < cnt; i++)    // 求每个质因数的次数
{
    int p = primes[i];
    sum[i] = get(a, p) - get(b, p) - get(a - b, p);
}

vector<int> res;
res.push_back(1);

for (int i = 0; i < cnt; i++)    // 用高精度乘法将所有质因子相乘
    for (int j = 0; j < sum[i]; j++)
        res = mul(res, primes[i]);

```

应用

```
using namespace std;

const int N = 5010;
int primes[N], cnt=0;
// v[i] 记录数字 i 为素数还是合数, v[i]=true时 i 为合数, 否则 i 为素数
bool v[N];
// sum[i]=c 表示质数 i 的个数为 c
int sum[N];

// 线性筛法
void get_primes(int n)
{
    for(int i=2; i<=n; ++i)
    {
        // i为质数, 则存在primes中
        if(!v[i]) primes[cnt++] = i;
        // 给当前数i乘上一个质因子pj
        for(int j=0; primes[j] <= n/i; ++j)
        {
            v[primes[j]*i] = true;
            if(i%primes[j]==0) break;
        }
    }
}

// 计算 n 里面含有质数 p 的个数, 这里的计算是不重不漏的。
// p^k的倍数会被计算k次: 第一次算p的倍数时, 被加一次; 第二次算p^2的倍数时, 被加一次; 第三次算p^3的倍数时, 被加一次... 第k次算p^k的倍数时, 被加一次。总共被加了k次, 是不重不漏的。
int get(int n, int p)
{
    int res=0;
    while(n)
    {
        res += n/p;
        n /= p;
    }
    return res;
}

// A * b: 把 b 看成一个整体, 然后与 A 中每一位相乘, A中的数字采用小端存储, 即低位数字存储在数组的前面, 高位数字存储在数组的后面
vector<int> mul(const vector<int>& A, const int b)
{
    if(b==0) return {0};
    vector<int> res;
    // t 表示乘法进位, 这里的进位不限于0 1, 可以为任意数字
    for(int i=0, t=0, n=A.size(); i<n || t>0; ++i)
    {
        // 获得当前位的乘积和
        if(i<n) t += A[i]*b;
        // 添加个位数字
```

```

        res.push_back(t%10);
        // 保留进位
        t/=10;
    }

    // 如 1234 * 0 = 0000, 需要删除前导0
    while(res.size()>1&&res.back()==0)res.pop_back();
    return res;
}

int main()
{
    int a,b;cin>>a>>b;

    // 将 a 分解质因数
    get_primes(a);

    for(int i=0;i<cnt;++i)
    {
        // 当前的质数为 p
        int p=primes[i];
        // 用分子里面 p 的个数减去分母里面 p 的个数。这里的计算组合数的公式为  $a!/(b!*(a-b)!)$ , 因此用 a
        // 里面 p 的个数减去 b 里面 p 的个数和 (a-b) 里面 p 的个数。
        sum[i]=get(a,p)-get(b,p)-get(a-b,p);
    }

    // 使用高精度乘法把所有质因子乘到一块去就好了
    vector<int> res={1};
    for(int i=0;i<cnt;++i)
        // res*p^k, 这里是k个p相乘, 不是k*p, 所以需要使用一个循环
        for(int j=0;j<sum[i];++j)
            res=mul(res,primes[i]);

    // 倒序打印 res 即可, 由于采用小端存储, 所以高位在后, 从后往前打印即可
    for(int i=res.size()-1;i>=0;i--)printf("%d",res[i]);
    return 0;
}

```

容斥原理应用

经典例题：[890. 能被整除的数 - AcWing题库]: <https://www.acwing.com/problem/content/892/>

AC代码：

```

using namespace std;
typedef long long LL;

const int N = 20;
int p[N], n, m;

int main() {
    cin >> n >> m;
    for(int i = 0; i < m; i++) cin >> p[i];
}

```

```

int res = 0;
//枚举从1 到 1111...(m个1)的每一个集合状态，(至少选中一个集合)
for(int i = 1; i < 1 << m; i++) {
    int t = 1;           //选中集合对应质数的乘积
    int s = 0;           //选中的集合数量

    //枚举当前状态的每一位
    for(int j = 0; j < m; j++){
        //选中一个集合
        if(i >> j & 1){
            //乘积大于n，则n/t = 0，跳出这轮循环
            if((LL)t * p[j] > n){
                t = -1;
                break;
            }
            s++;           //有一个1，集合数量+1
            t *= p[j];
        }
    }

    if(t == -1) continue;

    if(s & 1) res += n / t;           //选中奇数个集合，则系数应该是1，n/t为当前这种状态的集合数量
    else res -= n / t;              //反之则为 -1
}

cout << res << endl;
return 0;
}

```

详细题解: [AcWing 890. 能被整除的数 - AcWing]: <https://www.acwing.com/solution/content/29702/>

博弈论

NIM游戏

定理1: 必胜态的后继状态至少存在一个必败态

定理2: 必败态的后继状态均为必胜态

NIM游戏科普: [尼姆游戏 (学霸就是这样欺负人的) 哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1ek4y1q7JD/>

[再看nim游戏哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1nt4y1C7Sk/>

经典例题: [P2197 【模板】nim 游戏 - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)]: <https://www.luogu.com.cn/problem/P2197>

AC代码:

```

using namespace std;
int T;

```

```

int main() {
    cin >> T;
    while (T--) {
        int n;
        scanf("%d", &n);
        int ans = 0;
        for (int i = 0; i < n; i++) {
            int k;
            scanf("%d", &k);
            ans ^= k;
        }
        if (ans)
            puts("Yes");
        else
            puts("No");
    }
    return 0;
}

```

结论:

若初态为**必胜态**($a_1 \oplus a_2 \oplus \dots \oplus a_n \neq 0$).则先手必胜

若初态为**必败态**($a_1 \oplus a_2 \oplus \dots \oplus a_n = 0$).则先手必败

视频讲解: [581 尼姆 (Nim) 游戏【博弈论】哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1ns4y1D7dg/>

台阶型NIM游戏

经典例题: [892. 台阶-Nim游戏 - AcWing题库]: <https://www.acwing.com/problem/content/894/>

AC代码:

```

using namespace std;

const int N = 100010;

int main()
{
    int n;
    scanf("%d", &n);
    int res = 0;
    for (int i = 1; i <= n; i++) {
        int x;
        scanf("%d", &x);
        if (i & 1) res ^= x;
    }
    if (res) puts("Yes");
    else puts("No");

    return 0;
}

```

```
}
```

结论：若奇数台阶上的 $a_1 \oplus a_3 \oplus a_5 \oplus \dots \neq 0$ ，则先手必胜，反之先手必败。

视频讲解：[582 台阶型 Nim游戏【博弈论】哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV18M411M7T>
[C/](#)

集合型NIM游戏

经典例题：[893. 集合-Nim游戏 - AcWing题库]: <https://www.acwing.com/problem/content/895/>

AC代码：

```
using namespace std;

const int N=110,M=10010;
int n,m;
int f[M],s[N]; //s存储的是可供选择的集合,f存储的是所有可能出现过的情况的sg值

int sg(int x)
{
    if(f[x]!=-1) return f[x];
    //因为取石子数目的集合是已经确定了的,所以每个数的sg值也都是确定的,如果存储过了,直接返回即可
    unordered_set<int> S;
    //set代表的是有序集合(注:因为在函数内部定义,所以下一次递归中的S不与本次相同)
    for(int i=0;i<m;i++)
    {
        int sum=s[i];
        if(x>=sum) S.insert(sg(x-sum));
        //先延伸到终点的sg值后,再从后往前排查出所有数的sg值
    }
    for(int i=0;;i++)
    //循环完之后可以进行选出最小的没有出现的自然数的操作
    if(!S.count(i))
        return f[x]=i;
}

int main()
{
    cin>>m;
    for(int i=0;i<m;i++)
        cin>>s[i];

    cin>>n;
    memset(f,-1,sizeof(f)); //初始化f均为-1,方便在sg函数中查看x是否被记录过

    int res=0;
    for(int i=0;i<n;i++)
    {
        int x;
        cin>>x;
        res^=sg(x);
        //观察异或值的变化,基本原理与Nim游戏相同
    }
}
```

```

    }

    if(res) printf("Yes");
    else printf("No");

    return 0;
}

```

思路：转换成有向图游戏

视频讲解：[583 有向图游戏 SG函数【博弈论】哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1eT411B7A8/>

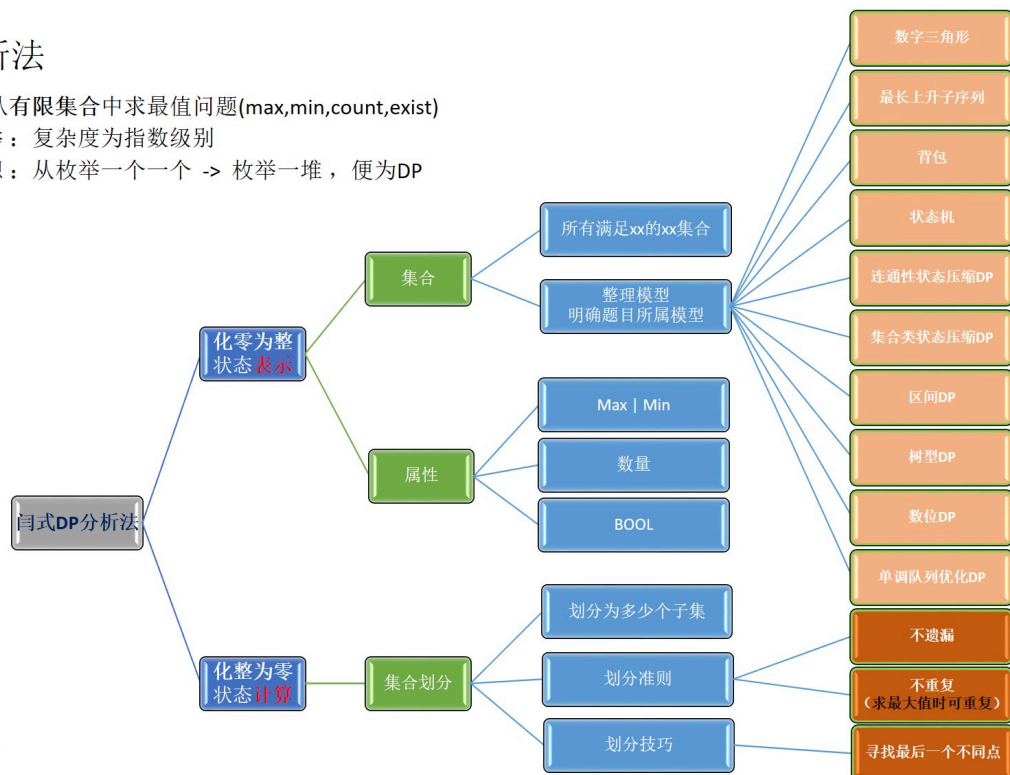
五、动态规划

动态规划三大特征：最优子结构、无后效性、重复子问题

无后效性：现在决定未来，未来与过去无关。

闫式DP分析法

- 动态规划：从有限集合中求最值问题(max,min,count,exist)
 - 暴力枚举：复杂度为指数级别
 - 优化思想：从枚举一个一个 -> 枚举一堆，便为DP



Edit : jasonlin

背包问题

01背包每件物品只能装一次

完全背包每件物品可以装无限次

多重背包每件物品只能装有限次（多次）

分组背包每组只能选择一件物品装入（01背包升级）

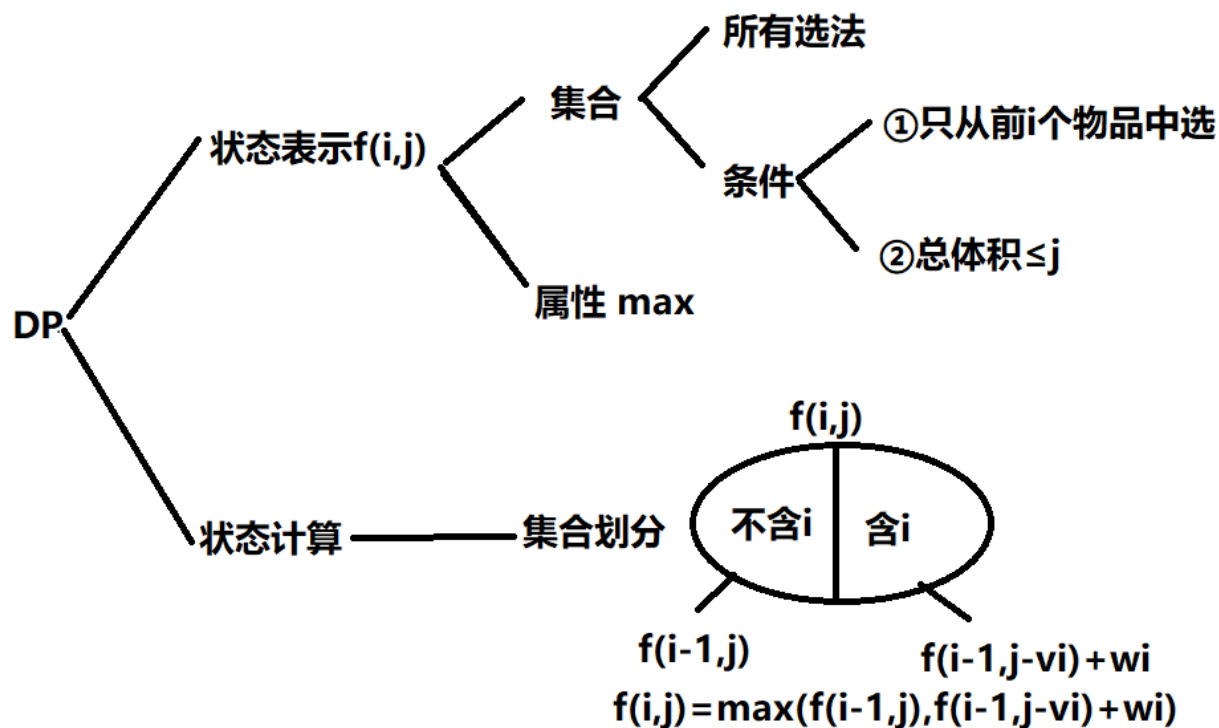
相关链接：<https://zhuanlan.zhihu.com/p/166439661>

01背包问题

01背包每件物品只能装一次

视频讲解: [408 背包DP【模板】 01背包哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1kp4y1e794/>

01背包DP分析



	A	B	C	D	E	F	G
1		0	1	2	3	4	5
2		0	0	0	0	0	0
3	1, 2	0	2	2	2	2	2
4	2, 4	0	2	4	6	6	6
5	3, 4	0	2	4	6	6	8
6	4, 5	0	2	4	6	6	8
7							

```
using namespace std;
const int N=1010;
int n,m;
int v[N],w[N]; //v代表体积, w代表价值
int f[N][N];

int main(){
    cin>>n>>m;
    for(int i=1;i<=n;i++)cin>>v[i]>>w[i];
    for(int i=1;i<=n;i++){ //i代表这n件物品
        {
            for(int j=1;j<=m;j++){ //j代表背包容量
                if(v[i]>j) //如果v[i]的容量大于当前的背包容量则不装进行下一个
```

```

        f[i][j]=f[i-1][j];
    else f[i][j]=max(f[i-1][j],f[i-1][j-v[i]]+w[i]); //如果v[i]的容量小于当前背包容量则可以
    以选择装与不装得到最大值
    }
}

cout<<f[n][m]<<endl; //输出最后的一个一定是最大的
return 0;
}

```

01背包，使用滚动数组，倒序遍历

```

using namespace std;
const int N=1010;
int n,m;
int v[N],w[N]; //v代表体积，w代表价值
int dp[N];

int main(){
    cin>>n>>m;
    for(int i=1;i<=n;i++)//

        i代表这n件物品
    {
        cin>>v[i]>>w[i]; //在线算法
        for(int j=m;j>=v[i];j--){ //j代表背包容量，滚动数组必须倒序遍历
            dp[j]=max(dp[j],dp[j-v[i]]+w[i]); //滚动数组
        }
    }
    cout<<dp[m]<<endl; //输出最后的一个一定是最大的
    return 0;
}

```

状态转移方程: $dp[j]=\max(dp[j],dp[j-v[i]]+w[i]);$

完全背包问题

完全背包每件物品可以装无限次

视频讲解: [409 背包DP 完全背包【动态规划】哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV15v411y7Qz/>

```

using namespace std;
int v[N],w[N];
int dp[N];
int main(){
    int n,m;
    cin>>n>>m;
    for(int i=1;i<=n;i++){ //遍历物品
        cin>>v[i]>>w[i]; //在线算法
        for(int j=v[i];j<=m;j++){ //正序遍历背包容量

```

```

        dp[j]=max(dp[j],dp[j-v[i]]+w[i]); //滚动数组
    }
}
cout<<dp[m]<<endl; //输出答案
return 0;
}

```

完全背包问题和01背包优化版的区别在于第二重循环的v[i]和m做交换

状态转移方程: $dp[j]=\max(dp[j], dp[j-v[i]]+w[i]);$

多重背包问题1

多重背包每件物品只能装有限次（多次）

```

using namespace std;
int n,m;
int v[N],w[N],s[N];
int dp[N][N];

int main(){
    cin>>n>>m;
    for(int i=1;i<=n;i++)cin>>v[i]>>w[i]>>s[i];
    for(int i=1;i<=n;i++) //物品
        for(int j=0;j<=m;j++) //背包容量
            for(int k=0;k<=s[i]&& k*v[i]<=j;k++)
                dp[i][j]=max(dp[i][j],dp[i-1][j-v[i]*k]+w[i]*k);
    cout<<dp[n][m]<<endl;
    return 0;
}

```

状态转移方程: $dp[i][j]=\max(dp[i][j], dp[i-1][j-v[i]*k]+w[i]*k);$ k为第i个物品的个数

多重背包问题2(二进制优化)

思路: 转换成2进制, 再用01背包求解

视频讲解: [410 背包DP 多重背包 二进制优化【动态规划】哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1MA41177cg/>

```

using namespace std;

const int N = 12010, M = 2010;

int n, m;
int v[N], w[N];
int f[M];

int main()
{
    cin >> n >> m;
}

```

```

int cnt = 0;
for (int i = 1; i <= n; i ++ )
{
    int a, b, s;
    cin >> a >> b >> s;
    int k = 1;
    while (k <= s)
    {
        cnt ++ ;
        v[cnt] = a * k;
        w[cnt] = b * k;
        s -= k;
        k *= 2;
    }
    if (s > 0)
    {
        cnt ++ ;
        v[cnt] = a * s;
        w[cnt] = b * s;
    }
} //二进制优化操作

n = cnt;

for (int i = 1; i <= n; i ++ )
    for (int j = m; j >= v[i]; j -- )
        f[j] = max(f[j], f[j - v[i]] + w[i]);

cout << f[m] << endl;

return 0;
}

```

分组背包问题

分组背包每组只能选择一件物品装入

视频讲解: [416 背包DP 分组背包【动态规划】哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV16a411w77X/>

```

using namespace std;
const int N=110;
int f[N];
int v[N][N],w[N][N],s[N];
int n,m,k;

int main(){
    cin>>n>>m;
    for(int i=0;i<n;i++){
        cin>>s[i];
        for(int j=0;j<s[i];j++){
            cin>>v[i][j]>>w[i][j];
        }
    }
}

```

```

    }

    for(int i=0;i<n;i++){
        for(int j=m;j>=0;j--){
            for(int k=0;k<s[i];k++){ //for(int k=s[i];k>=1;k--)也可以
                if(j>=v[i][k])
                    f[j]=max(f[j],f[j-v[i][k]]+w[i][k]);
            }
        }
    }
    cout<<f[m]<<endl;
}

```

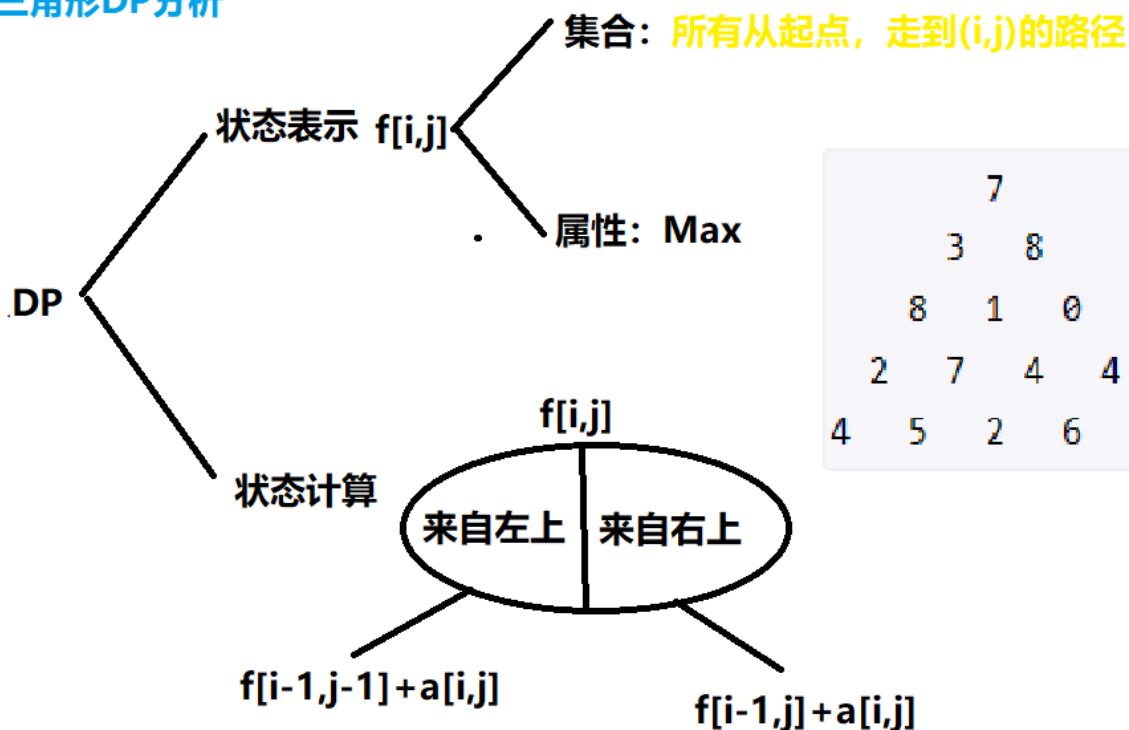
状态转移方程: $f[j]=\max(f[j],f[j-v[i][k]]+w[i][k]);$

线性DP

数字三角形

视频讲解: [402 线性DP 数字三角形【动态规划】 哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1Rk4y1173p/>

数字三角形DP分析



```

using namespace std;
const int N=510,INF=1e9;
int n;
int a[N][N];
int f[N][N];

int main(){

```

```

scanf("%d",&n);
for(int i=1;i<=n;i++){
    for(int j=1;j<=i;j++){
        scanf("%d",&a[i][j]);
    }
}
for(int i=0;i<=n;i++){
    for(int j=0;j<=i+1;j++){
        f[i][j]=-INF;
    }
}
f[1][1]=a[1][1];
for(int i=2;i<=n;i++){
    for(int j=1;j<=i;j++){
        f[i][j]=max(f[i-1][j-1]+a[i][j],f[i-1][j]+a[i][j]); //状态转移方程
    }
}
int res=-INF;
for(int i=1;i<=n;i++)res=max(res,f[n][i]);
printf("%d",res);
return 0;
}

```

状态转移方程: $f[i][j]=\max(f[i-1][j-1]+a[i][j], f[i-1][j]+a[i][j]);$

最长上升子序列1(LIS)

视频讲解: [403 线性DP 最长上升子序列【动态规划】 哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1K4y1e7t7/>

```

using namespace std;
const int N = 1010;

int n;
int a[N], f[N];

int main()
{
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) scanf("%d", &a[i]);
    for (int i = 1; i <= n; i++){
        f[i]=1; //只有a[i]一个数
        for (int j = 1; j <= i; j++)
            if(a[j]<a[i])
                f[i]=max(f[i], f[j]+1);
    }
    int res=0;
    for (int i = 1; i <= n; i++) res=max(res, f[i]);
    printf("%d\n", res);
    return 0;
}

```

状态转移方程: $\text{if}(a[j]<a[i])f[i]=\max(f[i], f[j]+1);$

最长上升子序列2(LIS二分优化)

视频讲解: [404 线性DP 最长上升子序列 二分优化哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1Kp4y1e77H/>

```
using namespace std;

const int N = 100010;

int n;
int a[N];
int q[N];

int main()
{
    scanf("%d", &n);
    for (int i = 0; i < n; i ++ ) scanf("%d", &a[i]);

    int len = 0;
    for (int i = 0; i < n; i ++ )
    {
        int l = 0, r = len;
        while (l < r)
        {
            int mid = l + r + 1 >> 1;
            if (q[mid] < a[i]) l = mid;
            else r = mid - 1;
        }
        len = max(len, r + 1);
        q[r + 1] = a[i]; // 替换或添加
    }

    printf("%d\n", len);

    return 0;
}
```

最长公共子序列(LCS)

视频讲解: [405 线性DP 最长公共子序列【动态规划】哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1EK411K7Eb/>

```
using namespace std;
const int N=1010;
int n,m;
char a[N],b[N];
int f[N][N];

int main()
{
    cin>>n>>m>>a+1>>b+1;
    for (int i = 1; i <= n; i ++ ){
```

```

        for (int j = 1; j <= m; j ++ ){
            f[i][j]=max(f[i-1][j],f[i][j-1]);
            if(a[i]==b[j])f[i][j]=max(f[i][j],f[i-1][j-1]+1);
        }
    }
    cout<<f[n][m]<<endl;
    return 0;
}

```

状态转移方程：

```

f[i][j]=max(f[i-1][j],f[i][j-1]);
if(a[i]==b[j])f[i][j]=max(f[i][j],f[i-1][j-1]+1);

```

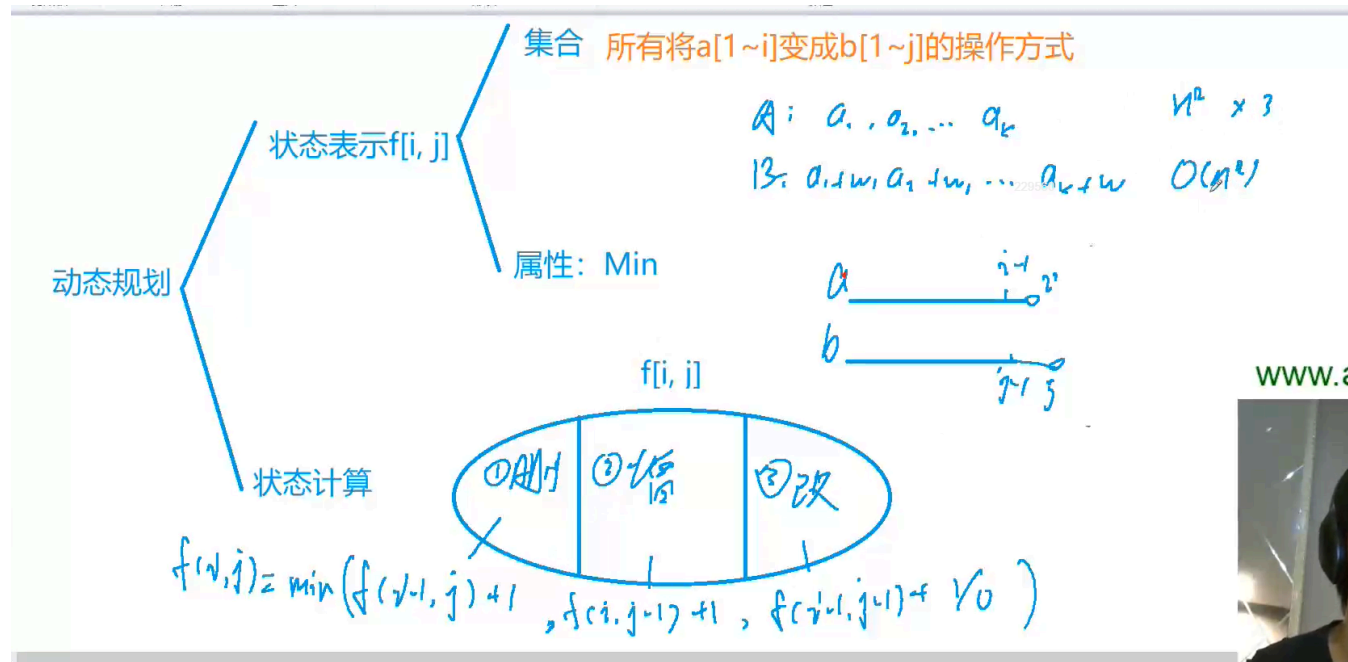
最短编辑距离

给定两个字符串 A 和 B，现在要将 A 经过若干操作变为 B，可进行的操作有：

1. 删除-将字符串 A 中的某个字符删除。
2. 插入-在字符串 A 的某个位置插入某个字符。
3. 替换-将字符串 A 中的某个字符替换为另一个字符。

现在请你求出，将 A 变为 B 至少需要进行多少次操作。

视频讲解：[407 线性DP 编辑距离【动态规划】 哔哩哔哩bilibili]： <https://www.bilibili.com/video/BV1gk4y117Z/>



```

using namespace std;
const int N = 1010;
int n, m;
char a[N], b[N];
int f[N][N];

int main()

```

```

{
    scanf("%d%s", &n, a+1);
    scanf("%d%s", &m, b+1);

    for (int i = 0; i <= m; i++) f[0][i]=i;
    for (int i = 0; i <= n; i++) f[i][0]=i; //初始化字符串的编辑操作
    for (int i = 1; i <= n; i++){
        for (int j = 1; j <= m; j++){
            f[i][j]=min(f[i-1][j]+1, f[i][j-1]+1);
            if(a[i]==b[j]) f[i][j]=min(f[i][j], f[i-1][j-1]);
            else f[i][j]=min(f[i][j], f[i-1][j-1]+1); //状态转移方程
        }
    }
    printf("%d\n", f[n][m]);
    return 0;
}

```

状态转移方程:

```

f[i][j]=min(f[i-1][j]+1, f[i][j-1]+1);
if(a[i]==b[j]) f[i][j]=min(f[i][j], f[i-1][j-1]);
else f[i][j]=min(f[i][j], f[i-1][j-1]+1); //状态转移方程

```

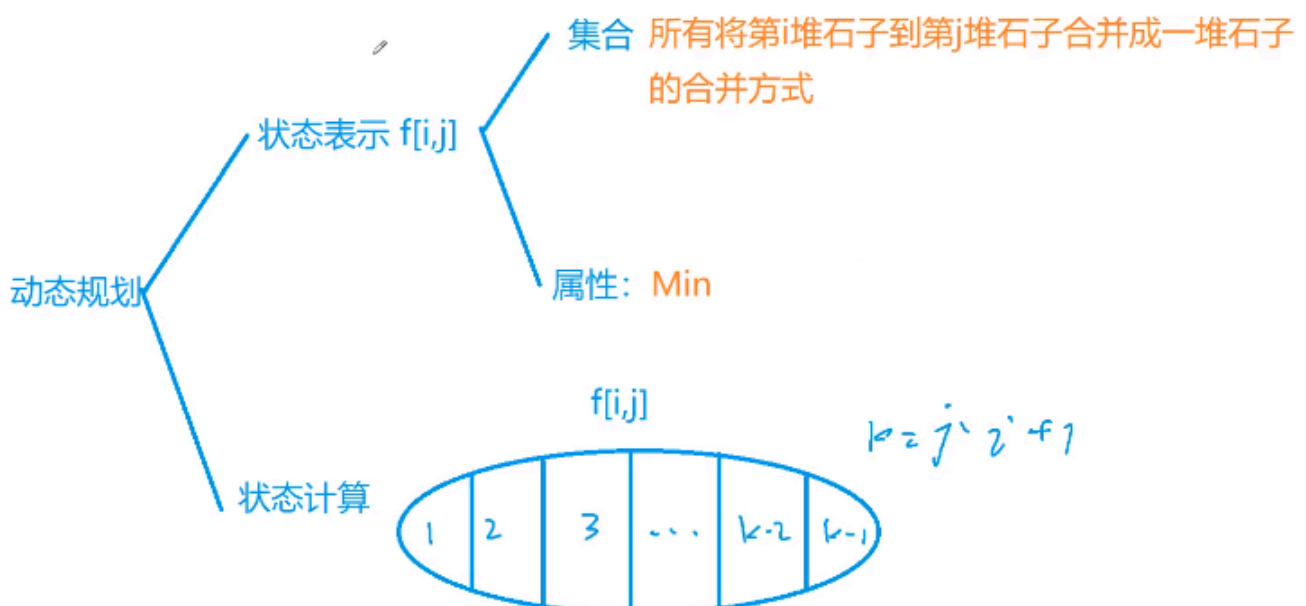
区间DP

石子合并

每堆石子有一定的质量，可以用一个整数来描述，现在要将这 N 堆石子合并成一堆。

每次只能合并相邻的两堆，合并的代价为这两堆石子的质量之和，合并后与这两堆石子相邻的石子将和新堆相邻，合并时由于选择的顺序不同，合并的总代价也不相同。

视频讲解: [428 区间DP【模板】石子合并哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1gz4y1y7Rv/>



```

using namespace std;
const int N = 310;

int n;
int s[N];
int f[N][N]; // 状态表示: 集合f[l][r]为[l,r]区间; 属性: 所堆成的最小值
int main()
{
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) scanf("%d", &s[i]);
    for (int i = 1; i <= n; i++) s[i] += s[i-1]; // 前缀和用来求一段区间的和

    for (int len = 2; len <= n; len++) // 区间长度为len//枚举长度
        for (int i = 1; i + len - 1 <= n; i++) { // 意思就是i在区间[1, n-len+1]中去//枚举区间
            int l = i, r = i + len - 1; // 区间在[i, i+len-1]中间长度为len//设置l和r的区间
            f[l][r] = 1e9; // 初始化最大值
            for (int k = l; k < r; k++) // 枚举分界点// 不取r
                f[l][r] = min(f[l][r], f[l][k] + f[k+1][r] + s[r] - s[l-1]); // 找到最小值状态转移方程为
            f[l][k] + f[k+1][r] + s[r] - s[l-1];
        }
    printf("%d\n", f[1][n]); // 输出区间[1,n]的最小值
    return 0;
}

```

状态转移方程 找到最小值状态转移方程为 $f[l][r] = \min(f[l][r], f[l][k] + f[k+1][r] + s[r] - s[l-1])$

计数类DP

整数划分

一个正整数 n 可以表示成若干个正整数之和，我们将这样的一种表示称为正整数 n 的一种划分。

现在给定一个正整数 n ，请你求出 n 共有多少种不同的划分方法。

完全背包写法

```

//完全背包的写法
using namespace std;
const int M = 1e9 + 7;
int f[1010], n;

int main()
{
    cin >> n;
    f[0] = 1;
    for (int i = 1; i <= n; i++)
        for (int j = i; j <= n; j++) {
            f[j] = (f[j-i] + f[j]) % M;
        }
    cout << f[n] << endl;
    return 0;
}

```

状态转移方程: $f[j]=f[j-1]+f[j]$

数位统计DP

计数问题

题目链接: [338. 计数问题 - AcWing题库]: <https://www.acwing.com/problem/content/340/>

求1~n中x的个数

设 $x=1$, $n=abcdefg$

分别求出1在每一位上出现的次数

例如求1在第4位上出现的次数

$1 \leq xxx1yyy \leq abcdefg$

(1)当 $xxx \in [0, abc-1]$ 时, 那么yyy就能取 $[000, 999]$, 即在4位出现的次数为 $abc \times 1000$

(如果 $x=0$ 的话, 因为不能出现前导零所以这里要特判一下, $xxx \in [1, abc-1]$ 要从最小为1开始计数)

(2)当 $xxx=abc$ 时, 有下三种情况

(2.1) $d < 1$, 即 $d=0$, 因为 $abc1yyy > abc0edg$, 所以在第4位出现的次数为0

(2.2) $d=1$, 那么 $yyy \in [000, efg]$, 所以在第4位出现的次数为 $efg+1$

(2.3) $d > 1$, 那么 $yyy \in [000, 999]$, 所以在第4位出现的次数为1000

综上所述: 把情况(1)和情况(2)累加起来就是求1~abcdefg中的第4位1出现的次数

```
using namespace std;

//因为我们举的分类中, 有要求一串数字中某个区间的数字, 例如abcdefg有一个分类要求出efg+1
int get(vector<int> num, int l, int r){
    int res=0;
    for(int i=l; i>=r; i--) res=res*10+num[i]; //这里从小到大枚举的是因为下面count的时候读入数据是从最低
    //为读到最高位, 那么此时在num里, 最高位存的就是数字的最低位, 那么假如我们要求efg, 那就是从2算到0
    return res;
}

int power10(int i) //这里有power10是因为有一个分类要求得十次方得值
{
    int res=1;
    while(i-->0) res*=10;
    return res;
}

int count(int n, int x){
    if(!n) return 0; //n=0则返回0
    vector<int> num; //num用来存储数中的每一位数字
    while(n){
        num.push_back(n%10);
        n/=10;
    }
    n=num.size(); //得出它的长度
    int res=0;
    for (int i = n-1; i >=0; i -- )
```

```

//这里需要注意，我们的长度需要减一，是因为num是从0开始存储，而长度是元素的个数，因此需要减1才能读到正确的数值，而!x出现的原因是因为我们不能让前导零出现，如果此时需要我们列举的是0得出现的次数，那么我们自然不能让他们出现第一位，而是从第二位开始枚举
{
    if(i<n-1)//其实这里可以不同if判断，因为for循环里面实际上就已经达成了if得判断，但为了方便理解还是加上if来理解，这里i要小于n-1的原因是因为我们不能越界只有7位数就最高从七位数开始读起
    {
        res+=get(num,n-1,i+1)*power10(i);//这里就是第一个分类，000~abc-1，那么此时情况个数就会是abc*103,这里的3取决于后面的efg的长度，假如他是efgh，那么就是4
        //这里的n-1,i+1,自己将数组列出然后根据分类标准就可以得出为什么l是n-1,r=i+1
        if(!x)res-=power10(i);//假如此时我们要列举的是0出现的次数，因为不能出现前导零，这样是不合法也不符合我们的分类情况，例如abcdefg我们列举d，那么他就得从001~abc-1，这样就不会直接到efg，而是会到0efg，因为前面不是前导零，自然就可以列举这个时候0出现的次数，所以要减掉1个power10
    }
    if(num[i]==x)res+=get(num,i-1,0)+1;
    else if(num[i]>x)res+=power10(i);
}
return res;//返回res，即出现次数
}

int main()
{
    int a,b;
    while(cin>>a>>b,a||b){
        if(a>b)swap(a,b);//a大于b则交换a，b使得变成合法参数
        for(int i=0;i<10;i++)
            cout<<count(b,i)-count(a-1,i)<<' '; //使用前缀和思想解决[a,b]的i出现的次数
        cout<<endl;
    }
    return 0;
}

```

状态压缩DP

蒙德里安的梦想

题目链接: [U204941 蒙德里安的梦想 - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)]: <https://www.luogu.com.cn/problem/U204941>

视频讲解: [431 状态压缩DP 蒙德里安的梦想【动态规划】哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1cv411b7EG/>

```

using namespace std;

const int N = 12,M=1<<N;

int n,m;
long long f[N][M];
bool st[M];

int main()
{

```

```

int n,m;
while(cin>>n>>m,n||m){
    memset(f, 0, sizeof f);
    //预处理：判断合并列的状态i是否合法
    //如果合并列的某行是1表示横放，是0表示竖放
    //如果合并列不存在连续的奇数个0，即为合法状态
    for (int i = 0; i < 1<<n; i ++ ){
        st[i]=true;
        int cnt=0;//记录合并列中连续0的个数
        for (int j = 0; j < n; j ++ ){
            if(i>>j&1){//如果是1
                if(cnt&1){//如果连续0的个数是奇数
                    st[i]=false;//记录i不合法
                    break;
                }
            }else cnt++;//如果是0，记录0的个数
        }
        if(cnt&1)st[i]=false;//处理高位0的个数
    }
    //状态计算
    f[0][0]=1;//第0列不横放是一种合法的方案
    for (int i = 1; i <= m; i ++ )//阶段：枚举列
        for (int j = 0; j < 1<<n; j ++ )//状态：枚举i列的状态
            for (int k = 0; k < 1<<n; k ++ )//状态：枚举i-1列的状态
                //两列状态兼容：不出现重叠的1，不出现连续奇数个0
                if((j&k)==0&&st[j|k])
                    f[i][j]+=f[i-1][k];
    cout<<f[m][0]<<endl;//第m列不横放，既答案
}
return 0;
}

```

状态转移方程：

```

if((j&k)==0&&st[j|k])
    f[i][j]+=f[i-1][k];

```

最短Hamilton路径

题目链接：[U122241 最短Hamilton路径 - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)]: <https://www.luogu.com.cn/problem/U122241>

```

using namespace std;

const int N = 20,M = 1 << N;

int n;
int w[N][N];
int f[M][N]; //第一维表示是否访问到该点的压缩状态，第二维是走到点j
              //f[i][j]表示状态为i并且到j的最短路径

int main(){

```

```

cin>>n;
for (int i = 0; i < n; i ++ )
    for (int j = 0; j < n; j ++ )//读入i到j的距离
        cin>>w[i][j];
memset(f, 0x3f, sizeof f);
f[1][0]=0;
for (int i = 0; i < 1 << n; i ++ )//枚举压缩的状态
    for (int j = 0; j < n; j ++ )//枚举到0~j的点
        if(i >> j & 1)//该状态存在j点
            for (int k = 0; k < n; k ++ )//枚举从j倒数第二个点k
                if(i >> k & 1)//倒数点k存在
                    f[i][j]=min(f[i][j],f[i-(1<<j)][k]+w[k][j]); //状态转移方程，在f[i][j]和
状态去掉j的点f[i-(i<<j)][k]+w[k][j]取最小值
cout<<f[(1<<n)-1][n-1]<<endl; //输出状态全满也就是所有点都经过且到最后一个点的最短距离
return 0;
}

```

状态转移方程：

```
f[i][j]=min(f[i][j],f[i-(1<<j)][k]+w[k][j]);
```

树形DP

没有上司的舞会

题目：[P1352 没有上司的舞会 - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)]: <https://www.luogu.com.cn/problem/P1352>

视频讲解：[417 树形DP 没有上司的舞会【动态规划】哔哩哔哩bilibili]: <https://www.bilibili.com/video/BV1eK411N7Ly/>

```

using namespace std;

const int N = 6010;

int n;
int w[N]; //每个节点的高兴度
int h[N], e[N], ne[N], idx; //邻接表存储树

bool st[N]; //判断是否有父节点
int f[N][2];

void add(int a, int b) // 添加一条边a->b
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx ++ ;
}

void dfs(int u){
    f[u][0]=0;
    f[u][1]=w[u]; //初始化f[u][1]，当第二维是0则不选该点即高兴度为0，同理f[u][1]=w[u];
    for (int i = h[u]; i != -1 ; i = ne[i] ){ //遍历u的子节点进行深度优先遍历
        int j=e[i];
        dfs(j);
    }
}

```

```

        //状态转移方程
        f[u][0] += max(f[j][0], f[j][1]); //f[u][0]表示不选择父节点u，所以在f[j][0]和f[j][1]取最大值
        f[u][1] += f[j][0]; //f[u][1]表示选择根节点u，所以累加不选择子节点的f[j][0]
    }
}

int main()
{
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> w[i];
    memset(h, -1, sizeof h);
    for (int i = 0; i < n-1; i++){
        int a, b;
        cin >> a >> b;
        add(b, a);
        st[a] = true; //存储是否存在父节点
    }
    int root = 1;
    while (st[root]) root++; //判断是否是根节点
    dfs(root); //dfs对f[i][j]进行状态转移计算
    cout << max(f[root][0], f[root][1]) << endl; //取选与不选根节点的最大值
    return 0;
}

```

状态转移方程：

```

f[u][0] += max(f[j][0], f[j][1]);
f[u][1] += f[j][0];

```

记忆化搜索

滑雪

题目链接：[P1434 [SHOI2002] 滑雪 - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)]: <https://www.luogu.com.cn/problem/P1434>

```

using namespace std;
const int N = 310;

int n, m;
int h[N][N];
int f[N][N];

int dx[4] = {-1, 0, 1, 0}, dy[4] = {0, 1, 0, -1};

int dp(int x, int y){
    int &v = f[x][y];
    if (v != -1) return v; //记忆化搜索核心
    v = 1;
    for (int i = 0; i < 4; i++){
        int a = x + dx[i], b = y + dy[i];
    }
}

```

```

        if(a>=1&&a<=n&&b>=1&&b<=m&&h[a][b]<h[x][y])//判断是否越界且上一个经过的点的高度是否大于当前
        高度
            v=max(v,dp(a,b)+1);
    }
    return v;
}

int main()
{
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; i ++ )
        for (int j = 1; j <= m; j ++ )
            scanf("%d", &h[i][j]);
    memset(f, -1, sizeof f);
    int res=0;
    for (int i = 1; i <= n; i ++ )
        for (int j = 1; j <= m; j ++ )
            res=max(res,dp(i,j));
    printf("%d\n",res);
    return 0;
}

```

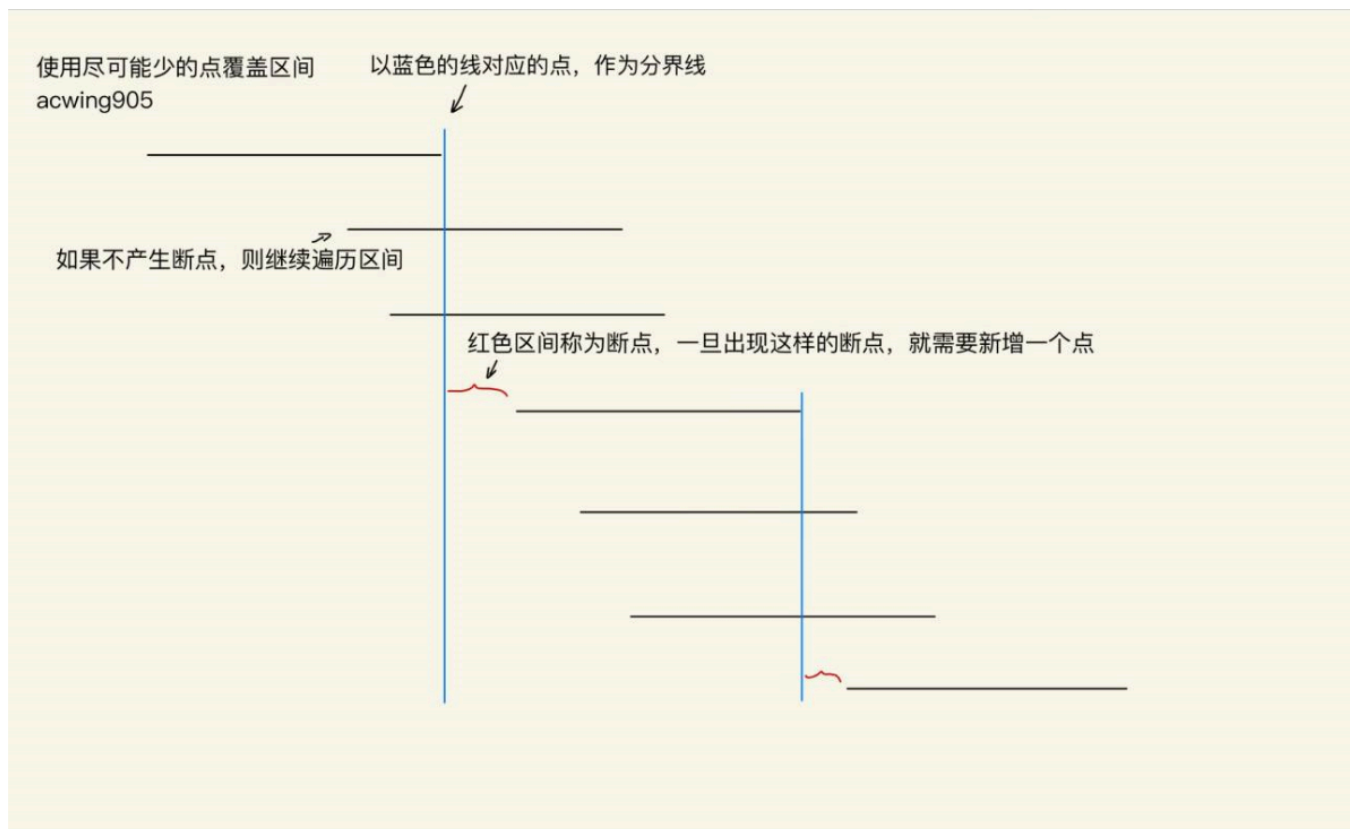
状态转移方程： $v = \max(v, dp(a, b) + 1);$

六、贪心

一个贪心算法总是做出当前最好的选择，也就是说，它期望通过局部最优选择从而得到全局最优的解决方案。---《算法导论》

区间问题

区间选点



给定 N 个闭区间 $[a_i, b_i]$ ，请在数轴上选择尽量少的点，使得每个区间内至少包含一个选出的点
输出选择的点的最小数量。

```
using namespace std;

const int N = 100010;

int n;
struct Range{
    int l,r;
    bool operator <(const Range& w)const{
        return r<w.r;
    } //重载小于号
}range[N];

int main()
{
    scanf("%d", &n);
    for (int i = 0; i < n; i ++ ){
        int l,r;
        scanf("%d%d", &l, &r);
        range[i]={l,r}; //读入l,r
    }
    sort(range,range+n); //按右端点进行排序
    int res=0,ed=-2e9; //ed代表上一个点的右端点
```

```

    for (int i = 0; i < n; i ++ ){
        if(range[i].l>ed){
            res++; //点的数量加一
            ed=range[i].r;
        }
    }
    printf("%d\n",res);
    return 0;
}

```

最大不相交区间数量

给定 N 个闭区间 $[a_i, b_i]$ ，请你在数轴上选择若干区间，使得选中的区间之间互不相交（包括端点）。

输出可选取区间的最大数量。

结论：最大不相交区间数量=最少覆盖区间点数

为什么最大不相交区间数=最少覆盖区间点数呢？

因为如果几个区间能被同一个点覆盖

说明他们相交了，所以有几个点就是有几个不相交区间

```

using namespace std;

const int N = 100010;

int n;
struct Range{
    int l,r;
    bool operator <(const Range& w)const{
        return r<w.r;
    }
}range[N];

int main()
{
    scanf("%d", &n);
    for (int i = 0; i < n; i ++ ){
        int l,r;
        scanf("%d%d", &l, &r);
        range[i]={l,r};
    }
    sort(range,range+n);
    int res=0,ed=-2e9;
    for (int i = 0; i < n; i ++ ){
        if(range[i].l>ed){
            res++;
            ed=range[i].r;
        }
    }
    printf("%d\n",res);
    return 0;
}

```

```
}
```

区间分组

1. 将所有区间按左端点从小到大排序

2. 从前往后处理每个区间

判断能否将其放到某个现有的组中 $L[i] > \text{Max}_r$

1. 如果不存在这样的组，则开新组，然后再将其放进去；

2. 如果存在这样的组，将其放进去，并更新当前组的 Max_r

```
using namespace std;

const int N = 1e5+10;

int n;
struct Range{
    int l,r;
    bool operator<(const Range &w)const{
        return l<w.l;
    } //按左端点排序
}range[N];

int main()
{
    scanf("%d", &n);
    for (int i = 0; i < n; i ++ ){
        int l,r;
        scanf("%d%d", &l, &r);
        range[i]={l,r};
    }
    sort(range,range+n); //sort排序
    priority_queue<int,vector<int>,greater<int>> heap; //小根堆维护所有组的右端点最小值
    for (int i = 0; i < n; i ++ ){ //从左往右枚举
        auto r=range[i]; //选择当前区间
        if(heap.empty() || heap.top()>=r.l)heap.push(r.r);
        else{
            heap.pop();
            heap.push(r.r);
        }
    }
    printf("%d\n",heap.size());
    return 0;
}
```

Huffman树

合并果子

在果园里，达达把打下的果子按种类分好堆后，打算将它们合成一堆。每次合并两堆果子，消耗的体力是这两堆果子重量之和，经过 $n - 1$ 次合并就能只剩一堆。达达合并果子时消耗的总体力是每次合并耗体力的总和，由于还要搬果子回家，所以要尽量节省体力。已知果子种类数和每种果子的数量，要设计出合并次序方案，让达达耗费体力最少，并输出这个最小体力耗费值。比如有 3 种果子，数量分别是 1、2、9，先合并数量为 1 和 2 的两堆，新堆数量是 3，耗体力 3，再把新堆和数量为 9 的那堆合并，新堆数量变成 12，耗体力 12，总共耗费体力就是 15，且能证明 15 是最小体力耗费值。

```
#include <iostream>
#include <queue>
#include <functional> // for greater<int>

using namespace std;

int main() {
    // 读取果子的种类数
    int n;
    cin >> n;

    // 使用小顶堆存储每种果子的数量，使用greater<int>确保堆顶元素最小
    priority_queue<int, vector<int>, greater<int>> pq;

    // 读取每种果子的数量，并将其加入小顶堆
    for (int i = 0; i < n; ++i) {
        int ai;
        cin >> ai;
        pq.push(ai);
    }

    // 用于存储总体的体力消耗
    long long total_cost = 0;

    // 当堆中的果子数量大于1时，继续合并
    while (pq.size() > 1) {
        // 取出堆顶的两个最小元素
        int first = pq.top();
        pq.pop();
        int second = pq.top();
        pq.pop();

        // 合并两个最小堆，计算体力消耗
        int merged = first + second;
        total_cost += merged;

        // 将合并后的结果放回堆中
        pq.push(merged);
    }

    // 输出最终的体力消耗值
    cout << total_cost << endl;
```

```
    return 0;
}
```

排序不等式

排队打水

有 n 个人排队到 1 个水龙头处打水，第 i 个人装满水桶所需的时间是 t_i ，请问如何安排他们的打水顺序才能使所有人的等待时间之和最小？

$t[i]$ 从小到大排序

计算公式： $t[0] \times (n-1) + t[1] \times (n-2) + t[2] \times (n-3) \dots + t[n] \times 0$

```
using namespace std;
typedef long long LL;
const int N = 1e5 + 10;
int t[N];

int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &t[i]);
    sort(t, t + n); // 排序
    LL ans = 0;
    for (int i = 0; i < n; i++) {
        ans += t[i] * (n - i - 1); // 计算
    }
    printf("%lld", ans);
    return 0;
}
```

七、杂项

提高课模板

这里为算法提高课的模板内容，算法提高课的模板较少不好整理，所以专门放在杂项里面。

高级数据结构—AC自动机

[AcWing 1282. 搜索关键词----AC自动机模板题(KMP + trie) - AcWing]: <https://www.acwing.com/solution/content/50169/>

视频讲解: [F08【模板】AC自动机——信息学竞赛算法]: <https://www.bilibili.com/video/BV1tF41157Dy/>

模板

```
void insert(char str[]) // 将str插入Trie中
{
    int p = 0;
    for (int i = 0; str[i]; i++)
```

```

    {
        int u = str[i] - 'a';
        if (!tr[p][u]) tr[p][u] = ++ idx;
        p = tr[p][u];
    }
    cnt[p] ++ ; // 记录单词出现次数
}

void build() // 创建AC自动机
{
    int hh = 0, tt = -1;
    for (int i = 0; i < 26; i ++ )
        if (tr[0][i])
            q[ ++ tt] = tr[0][i];
    while (hh <= tt)
    {
        int t = q[hh ++ ];
        for (int i = 0; i < 26; i ++ )
        {
            int p = tr[t][i];
            if (!p) tr[t][i] = tr[ne[t]][i];
            else
            {
                ne[p] = tr[ne[t]][i];
                cnt[p] += cnt[ne[p]];
                q[ ++ tt] = p;
            }
        }
    }
}

```

应用

题目：[1282. 搜索关键词 - AcWing题库]:<https://www.acwing.com/problem/content/1284/>

```

#include<iostream>
#include<cstring>
#include<algorithm>
using namespace std;
const int N = 10010, S = 55, M = 1000010;
int trie[N * S][26], cnt[N * S], idx; //cnt[i]表示以i + 'a'为结尾的个数    idx为当前树节点的指针
char str[M]; //以"/0"为结尾，所以不用每次都更新
int que[N * S], fail[N * S]; //que[]表示队列，fail[]为失配指针(下标表示树节点的指针)
int n;

void insert() {
    int p = 0;
    for (int i = 0; str[i]; ++i) {
        int u = str[i] - 'a';
        if (!trie[p][u]) trie[p][u] = ++idx;
        p = trie[p][u];
    }
    cnt[p]++;
}

```

```

}

void build() { //构造fail数组, bfs
    int hh = 0, tt = -1; //队头和队尾指针
    //根节点是第0层
    for (int i = 0; i < 26; ++i) { //第一层的元素全部入队
        if (trie[0][i]) que[++tt] = trie[0][i];
    }
    while (hh <= tt) {
        int ans = que[hh++];
        //枚举当前队头的26个分支
        for (int i = 0; i < 26; ++i) {
            if (trie[ans][i]) { //如果存在我们就让它的fail指针指向他父亲节点 a 的 fail 指针指向的那个
                //节点（根）的具有相同字母的子节点
                fail[trie[ans][i]] = trie[fail[ans]][i];
                que[++tt] = trie[ans][i]; //当前节点入队
            } else { //就算不存在，不跳，他的值等于父节点的fail只想的具有相同字母的子节点
                trie[ans][i] = trie[fail[ans]][i];
            }
        }
    }
}

int main() {
    int t;
    cin >> t;
    while (t--) {
        memset(cnt, 0, sizeof cnt);
        memset(trie, 0, sizeof trie);
        memset(fail, 0, sizeof fail);
        idx = 0;
        cin >> n;
        for (int i = 0; i < n; ++i) {
            scanf("%s", str);
            insert();
        }

        build();
        scanf("%s", str);

        int res = 0;
        //j记录当前树节点的指针，初始是根节点
        for (int i = 0, j = 0; str[i]; ++i) { //枚举总串str的每一个字母
            int u = str[i] - 'a';
            j = trie[j][u]; //跳到下一个树节点
            int p = j; //每次从当前树节点开始

            //fail[p]所指向的树节点如果有结尾标记可以直接算上，因为当前模式串后缀和fail指针指向的模式串部
            //分前缀相同，所以是包含在里面的
            while (p) { //假如模式串"she"可以匹配上，那么匹配到"e"的时候，用fail指针跳到模式
                //串"he"的"e"，那么也一定能够匹配"he"
                res += cnt[p];
                cnt[p] = 0; //去除标记
            }
        }
    }
}

```

```

        p = fail[p];
    }
}
cout << res << endl;
}
return 0;
}

```

高级数据结构—树状数组

[楼兰图腾题解 - AcWing]: <https://www.acwing.com/solution/content/13818/>

视频讲解: [C81【模板】树状数组 点修+区查 区修+点查]: <https://www.bilibili.com/video/BV17N4y1x7c6/>

模板

```

int lowbit(int x)
{
    return x & -x;
}

void update(int x, int c) // 位置x加c
{
    for (int i = x; i <= n; i += lowbit(i)) tr[i] += c;
}

int query(int x) // 返回前x个数的和
{
    int res = 0;
    for (int i = x; i; i -= lowbit(i)) res += tr[i];
    return res;
}

```

应用

[241. 楼兰图腾 - AcWing题库]: <https://www.acwing.com/problem/content/243/>

```

#include <iostream>
#include <cstdio>
#include <cstring>

using namespace std;

const int N = 2000010;

typedef long long LL;

int n;
//t[i]表示树状数组i结点覆盖的范围和
int a[N], t[N];
//Lower[i]表示左边比第i个位置小的数的个数
//Greater[i]表示左边比第i个位置大的数的个数

```

```

int Lower[N], Greater[N];

//返回非负整数x在二进制表示下最低位1及其后面的0构成的数值
int lowbit(int x)
{
    return x & -x;
}

//将序列中第x个数加上k。
void update(int x, int k)
{
    for(int i = x; i <= n; i += lowbit(i)) t[i] += k;
}

//查询序列前x个数的和
int query(int x)
{
    int sum = 0;
    for(int i = x; i; i -= lowbit(i)) sum += t[i];
    return sum;
}

int main()
{
    scanf("%d", &n);
    for(int i = 1; i <= n; i++) scanf("%d", &a[i]);

    //从左向右，依次统计每个位置左边比第i个数y小的数的个数、以及大的数的个数
    for(int i = 1; i <= n; i++)
    {
        int y = a[i]; //第i个数

        //在前面已加入树状数组的所有数中统计在区间[1, y - 1]的数字的出现次数
        Lower[i] = query(y - 1);

        //在前面已加入树状数组的所有数中统计在区间[y + 1, n]的数字的出现次数
        Greater[i] = query(n) - query(y);

        //将y加入树状数组，即数字y出现1次
        update(y, 1);
    }

    //清空树状数组，从右往左统计每个位置右边比第i个数y小的数的个数、以及大的数的个数
    memset(t, 0, sizeof t);

    LL resA = 0, resV = 0;
    //从右往左统计
    for(int i = n; i >= 1; i--)
    {
        int y = a[i];
        resA += (LL)Lower[i] * query(y - 1);
        resV += (LL)Greater[i] * (query(n) - query(y));
    }
}

```

```

        //将y加入树状数组，即数字y出现1次
        update(y, 1);
    }

    printf("%lld %lld\n", resv, resA);

    return 0;
}

```

高级数据结构—线段树

[浅谈线段树 - AcWing]: https://www.acwing.com/file_system/file/content/whole/index/content/6505356/

[AcWing 1275. 最大数题解 - AcWing]: <https://www.acwing.com/solution/content/61919/>

视频讲解: [C02 【模板】线段树+懒标记 Luogu P3372 线段树]: <https://www.bilibili.com/video/BV1G34y1L7b3/>

模板

```

struct Node
{
    int l, r;
    // TODO: 需要维护的信息和懒标记
}tr[N * 4];

void pushup(int u)
{
    // TODO: 利用左右儿子信息维护当前节点的信息
}

void pushdown(int u)
{
    // TODO: 将懒标记下传
}

void build(int u, int l, int r)
{
    if (l == r) tr[u] = {l, r};
    else
    {
        tr[u] = {l, r};
        int mid = l + r >> 1;
        build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
        pushup(u);
    }
}

void update(int u, int l, int r, int d)
{
    if (tr[u].l >= l && tr[u].r <= r)
    {
        // TODO: 修改区间
    }
}

```

```

    }
    else
    {
        pushdown(u);
        int mid = tr[u].l + tr[u].r >> 1;
        if (l <= mid) update(u << 1, l, r, d);
        if (r > mid) update(u << 1 | 1, l, r, d);
        pushup(u);
    }
}

int query(int u, int l, int r)
{
    if (tr[u].l >= l && tr[u].r <= r)
    {
        return ; // TODO 需要补充返回值
    }
    else
    {
        pushdown(u);
        int mid = tr[u].l + tr[u].r >> 1;
        int res = 0;
        if (l <= mid ) res = query(u << 1, l, r);
        if (r > mid) res += query(u << 1 | 1, l, r);
        return res;
    }
}

```

应用

[1275. 最大数 - AcWing题库]: <https://www.acwing.com/problem/content/1277/>

```

#include<iostream>
using namespace std;

const int N = 2e5 + 5;
typedef long long LL;

//线段树的结点，最大空间开4倍
struct Node{
    int l, r;
    int v; //最大值
}tr[N * 4];

int m, p;

//u为当前线段树的结点编号
void build(int u, int l, int r) {
    tr[u] = {l, r};
    if(l == r) return;
    int mid = l + r >> 1;
    build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
}

```

```

//查询以u为根节点，区间[l, r]中的最大值
int query(int u, int l, int r) {
    //      Tl-----Tr
    //      L-----R
    //1.不必分治，直接返回
    if(tr[u].l >= l && tr[u].r <= r) return tr[u].v;

    int mid = tr[u].l + tr[u].r >> 1;
    int v = 0;
    //      Tl-----m-----Tr
    //      L-----R
    //2.需要在tr的左区间[Tl, m]继续分治
    if(l <= mid) v = query(u << 1, l, r);

    //      Tl-----m-----Tr
    //      L-----R
    //3.需要在tr的右区间(m, Tr]继续分治
    if(r > mid) v = max(v, query(u << 1 | 1, l, r));

    //      Tl-----m-----Tr
    //      L-----R
    //2.3涵盖了这种情况
    return v;
}

//u为结点编号，更新该区间的最大值
void modify(int u, int x, int v) {
    if(tr[u].l == tr[u].r) tr[u].v = v; //叶节点，递归出口
    else {
        int mid = tr[u].l + tr[u].r >> 1;
        //分治处理左右子树，寻找x所在的子树
        if(x <= mid) modify(u << 1, x, v);
        else modify(u << 1 | 1, x, v);
        //回溯，拿子节点的信息更新父节点，即pushup操作
        tr[u].v = max(tr[u << 1].v, tr[u << 1 | 1].v);
    }
}

int main()
{
    //n表示树中的结点个数，last保存上一次查询的结果
    int n = 0, last = 0;
    cin >> n >> p;

    //初始化线段树，结点的区间最多为[1, m]。
    build(1, 1, n);

    while(m--)
    {
        char op;
        cin >> op;
        if(op == 'A') //添加结点

```

```

{
    int t;
    cin >> t;
    //在n + 1处插入
    modify(1, n + 1, ((LL)t + last) % p);
    //结点个数+1
    n++;
}
else
{
    int L;
    cin >> L;
    //查询[n - L + 1, n]内的最大值, u = 1, 即从根节点开始查询
    last = query(1, n - L + 1, n);
    cout << last << endl;
}
}

return 0;
}

```

图论—倍增求LCA(最近公共祖先)

[AcWing 1172. 祖孙询问(树上倍增LCA)题解 - AcWing]: <https://www.acwing.com/solution/content/20554/>

视频讲解:

[D09 倍增算法 P3379 【模板】最近公共祖先 (LCA)]: <https://www.bilibili.com/video/BV1vg41197Xh/>

[动画讲解速通LCA | 倍增思想 | 最近公共祖先]: <https://www.bilibili.com/video/BV18B66Y6E3T>

模板

```

void bfs(int root) // 预处理倍增数组
{
    memset(depth, 0x3f, sizeof depth);
    depth[0] = 0, depth[root] = 1; // depth存储节点所在层数
    int hh = 0, tt = 0;
    q[0] = root;
    while (hh <= tt)
    {
        int t = q[hh ++ ];
        for (int i = h[t]; ~i; i = ne[i])
        {
            int j = e[i];
            if (depth[j] > depth[t] + 1)
            {
                depth[j] = depth[t] + 1;
                q[ ++ tt] = j;
                fa[j][0] = t; // j的第二次幂个父节点
                for (int k = 1; k <= 15; k ++ )
                    fa[j][k] = fa[fa[j][k - 1]][k - 1];
            }
        }
    }
}

```

```

    }
}

int lca(int a, int b) // 返回a和b的最近公共祖先
{
    if (depth[a] < depth[b]) swap(a, b);
    for (int k = 15; k >= 0; k -- )
        if (depth[fa[a][k]] >= depth[b])
            a = fa[a][k];
    if (a == b) return a;
    for (int k = 15; k >= 0; k -- )
        if (fa[a][k] != fa[b][k])
        {
            a = fa[a][k];
            b = fa[b][k];
        }
    return fa[a][0];
}

```

应用

[1172. 祖孙询问 - AcWing题库]: <https://www.acwing.com/problem/content/description/1174/>

```

#include<iostream>
#include<cstring>
#include<algorithm>
#include<queue>

using namespace std;

const int N = 40010, M = N * 2;

int n, m;
int h[N], e[M], ne[M], idx;
int depth[N], fa[N][16]; // 往上跳2^k步后的父亲节点
int q[N];

void add(int a, int b)
{
    e[idx] = b;
    ne[idx] = h[a];
    h[a] = idx ++;
}

void bfs(int root) // 宽搜不容易因为递归层数过多爆栈
{
    memset(depth, 0x3f, sizeof depth);
    // 哨兵depth[0] = 0: 如果从i开始跳2^j步会跳过根节点
    // fa[fa[j][k-1]][k-1] = 0
    // 那么fa[i][j] = 0 depth[fa[i][j]] = depth[0] = 0
    depth[0] = 0, depth[root] = 1;
    queue<int> q;
    q.push(root);
}

```

```

while(q.size())
{
    int t = q.front();
    q.pop();
    for(int i=h[t];i!=-1;i=ne[i])
    {
        int j = e[i];
        if(depth[j]>depth[t]+1)//说明j还没被搜索过
        {
            depth[j] = depth[t]+1;
            q.push(j);//把第depth[j]层的j加进队列
            fa[j][0] = t;//j往上跳2^0步后就是t
            for(int k=1;k<=15;k++)
            {
                fa[j][k] = fa[fa[j][k-1]][k-1];
            }
        }
    }
}

int lca(int a, int b)
{
    // 为方便处理 当a在b上面时 把a b 互换
    if (depth[a] < depth[b]) swap(a, b);
    //把深度更深的a往上跳到b
    for (int k = 15; k >= 0; k -- )
        //当a跳完2^k依然在b下面 我们就一直跳
        //二进制拼凑法
        //这里因为
        if (depth[fa[a][k]] >= depth[b])
            a = fa[a][k];
    //如果跳到了b
    if (a == b) return a;
    //a,b同层但不同节点
    for (int k = 15; k >= 0; k -- )
        // 假如a,b都跳出根节点,fa[a][k]==fa[b][k]==0 不符合更新条件
        if (fa[a][k] != fa[b][k])
        {
            a = fa[a][k];
            b = fa[b][k];
        }
    //循环结束 到达lca下一层
    //lca(a,b) = 再往上跳1步即可
    return fa[a][0];
}

int main()
{
    cin >> n;
    int root = 0;
    memset(h, -1, sizeof h);

```

```

for (int i = 0; i < n; i ++ )
{
    int a, b;
    cin >> a >> b;
    if (b == -1) root = a;
    else add(a, b), add(b, a);
}

bfs(root); //建fa[i][j]

scanf("%d", &m);
while (m -- )
{
    int a, b;
    cin >> a >> b;
    int p = lca(a, b);
    if (p == a) cout << "1" << endl;
    else if (p == b) cout << "2" << endl;
    else cout << "0" << endl;
}
return 0;
}

```

洛谷题单

能力全面提升综合题单 - 题单<https://www.luogu.com.cn/training/9391>

李煜东《算法竞赛进阶指南》题单<https://www.luogu.com.cn/training/400>

由数据范围反推算法复杂度以及算法内容

一般ICPC或者笔试题的时间限制是1秒或2秒。

在这种情况下，C++代码中的操作次数控制在 $10^7 \sim 10^8$ 为最佳。

下面给出在不同数据范围下，代码的时间复杂度和算法该如何选择：

1. $n \leq 30$, 指数级别, dfs+剪枝, 状态压缩dp
2. $n \leq 100 \Rightarrow O(n^3)$, floyd, dp, 高斯消元
3. $n \leq 1000 \Rightarrow O(n^2)$, $O(n^2 \log n)$, dp, 二分, 朴素版Dijkstra、朴素版Prim、Bellman-Ford
4. $n \leq 10000 \Rightarrow O(n * \sqrt{n})$, 块状链表、分块、莫队
5. $n \leq 100000 \Rightarrow O(n \log n) \Rightarrow$ 各种sort, 线段树、树状数组、set/map、heap、拓扑排序、dijkstra+heap、prim+heap、Kruskal、spfa、求凸包、求半平面交、二分、CDQ分治、整体二分、后缀数组、树链剖分、动态树
6. $n \leq 1000000 \Rightarrow O(n)$, 以及常数较小的 $O(n \log n)$ 算法 $\Rightarrow$ 单调队列、hash、双指针扫描、BFS、并查集、kmp、AC自动机, 常数比较小的 $O(n \log n)$ 的做法: sort、树状数组、heap、dijkstra、spfa
7. $n \leq 10000000 \Rightarrow O(n)$, 双指针扫描、kmp、AC自动机、线性筛素数
8. $n \leq 10^9 \Rightarrow O(\sqrt{n})$, 判断质数
9. $n \leq 10^{18} \Rightarrow O(\log n)$, 最大公约数, 快速幂, 数位DP
10. $n \leq 10^{1000} \Rightarrow O((\log n)^2)$, 高精度加减乘除

11. $n \leq 10^{100000} \Rightarrow O(\log k \times \log \log k)$, k 表示位数, 高精度加减、FFT/NTT

此笔记适用于AcWing网站的算法基础课, 所有的资源链接、代码模板全部来源于网络, 这个文档只是做了一些收集和整理, 感谢文档中的所有资源原作者们!

笔记作者QQ: 2468197060

笔记QQ群聊: 1021549627

欢迎一起交流技术